

Algoritmos

De la lógica a la programación

Contenido

1	Créditos Editoriales	1
2	Acerca de este volumen	3
3	Presentación del curso	5
3.1	Objetivo general de la asignatura	5
3.2	Aprendizajes esperados	5
4	¿Qué es un algoritmo?	7
4.1	Elementos básicos de un algoritmo	7
4.2	Ejemplo cotidiano	7
5	¿Cómo desarrollar un algoritmo?	9
6	¿Cómo se describe un algoritmo?	11
7	Variables: de neuronas a espacios de memoria	13
7.1	Algoritmo para sumar dos números	13
7.2	Pseudocódigo	14
7.3	Explicación paso a paso	14
8	Ejercicio guiado: multiplicar tres números	15
8.1	Análisis	15
8.2	Pseudocódigo	15
9	Diagramas de flujo	17
9.1	Nomenclatura básica	17
9.2	Ejemplo gráfico del flujo para sumar dos números	18
9.3	Ejemplo textual del flujo para sumar dos números	18
10	Bifurcaciones y decisiones	21
10.1	Ejemplo: mayor de edad	21
10.2	Diagrama de flujo	22
10.3	Flujo lógico	22
11	Operadores	25
12	Operadores aritméticos	27
12.1	Módulo o residuo	27

13 Operadores relacionales	29
14 Operadores lógicos	31
14.1 Operador Y: &&	31
14.2 Operador O: 	31
14.3 Operador NO: !	32
15 Casos de análisis	33
16 Caso 1: operador &&	35
17 Caso 2: combinación de y &&	37
18 Caso 3: operador !	39
19 Prioridad de operadores	41
20 Introducción a ciclos	43
20.1 ¿Cuándo se aplican los ciclos?	43
21 Ejemplo: sumar N números	45
21.1 Análisis	45
21.2 Diagrama de flujo	46
21.3 Pseudocódigo	47
22 Ejemplo en C++: sumar del 1 hasta N	49
23 Ejemplo en C++: encontrar el mayor de N números	51
24 Programa de aplicación: ecuación de primer grado	53
24.1 Diagrama de flujo	54
24.2 Pseudocódigo	54
24.3 Código en C++	55
25 Ejercicios para resolver en clase	57
25.1 Ejercicio 1	57
25.2 Ejercicio 2	57
25.3 Ejercicio 3	57
25.4 Ejercicio 4	57
25.5 Ejercicio 5	57
25.6 Ejercicio 6	57
26 Resumen de la clase	59
27 Tarea sugerida	61
28 Cierre	63
29 Clase 2. Primeros pasos en C++	65
29.1 Objetivos	65
29.2 Herramienta de trabajo	65

29.3 Nuestro primer programa	65
29.3.1 Explicación	66
29.4 Entrada y salida	66
29.5 Tipos de datos	66
29.6 Ejercicio guiado	66
29.7 Laboratorio	67
29.7.1 Ejercicio 1	67
29.7.2 Ejercicio 2	67
29.7.3 Ejercicio 3	67
29.7.4 Ejercicio 4	67
29.8 Introducción a <code>while</code>	67
29.9 Resumen	68
29.10 Tarea	68
30 Laboratorio 1. Desarrollo de una solución informática	69
31 Laboratorio 1	71
31.1 Desarrollo de una solución informática	71
31.1.1 Objetivo	71
32 Requerimiento	73
33 Actividades a desarrollar	75
33.1 Parte 1. Análisis del requerimiento	75
33.1.1 Entradas	75
33.1.2 Procesos	75
33.1.3 Salidas	76
34 Parte 2. Diseño del algoritmo	77
34.1 Pseudocódigo	77
34.2 Diagrama de flujo	77
35 Parte 3. Implementación	79
36 Parte 4. Casos de prueba	81
36.1 Caso de prueba No. 1	81
36.2 Caso de prueba No. 2	81
37 Evidencia	83
38 Criterios de evaluación	85
39 Conclusión	87
40 Capítulo 3. Expresiones lógicas y estructuras condicionales	89
40.1 Presentación de la clase	89
40.2 Distribución de los 80 minutos	89
41 Objetivos de aprendizaje	91

42 Repaso rápido: expresiones y operadores	93
42.1 Operadores aritméticos	93
42.2 Operadores relacionales	93
42.3 Operadores lógicos	94
42.4 Precedencia simplificada	94
43 Estructura <code>if...else if...else</code>	95
43.1 Sintaxis	95
44 Ejercicio guiado 1: calculadora con <code>if</code>	97
44.1 Requerimiento	97
44.2 Parte A. Análisis	97
44.2.1 Entradas	97
44.2.2 Procesos	97
44.2.3 Salidas	98
44.3 Parte B. Diagrama de flujo	99
44.4 Parte C. Pseudocódigo	100
44.5 Parte D. Código C++	100
45 El mismo ejercicio con <code>switch...case</code>	103
45.1 Sintaxis	103
45.2 Código C++	103
46 Comparación entre <code>if</code> y <code>switch</code>	105
47 Ejercicio guiado 2: clasificación de edad	107
47.1 Requerimiento	107
47.2 Diagrama de flujo	108
47.3 Código C++	109
48 Actividad rápida en parejas	111
49 Errores frecuentes	113
49.1 Usar <code>=</code> en lugar de <code>==</code>	113
49.2 Colocar condición después de <code>else</code>	113
49.3 Olvidar <code>break</code>	113
49.4 Usar <code>switch</code> para rangos	113
50 Resumen de la clase	115
51 Tarea	117
51.1 Problema 1. Menú de conversión	117
51.1.1 Entregables	117
51.2 Problema 2. Clasificación de notas	117
51.2.1 Validación	118
51.2.2 Entregables	118
52 Cierre	119

53 Laboratorio No. 3: ciclos, <code>enum</code> y simulación de un cajero automático	121
53.1 Propósito del laboratorio	121
53.2 Ruta de trabajo	121
53.3 Recordatorio rápido	122
53.3.1 Estructura secuencial	122
53.3.2 Decisión con <code>if</code>	122
53.3.3 Selección múltiple con <code>switch</code>	123
53.3.4 Repetición con <code>while</code>	123
53.4 Ciclo <code>for</code>	124
53.4.1 Ejercicio 1: mostrar los números del 1 al 10	124
53.4.2 Ejercicio 2: tabla de multiplicar	124
53.5 Uso de <code>enum</code>	125
53.5.1 Ventaja principal	126
53.6 Proyecto del laboratorio: cajero automático	127
53.6.1 Situación	127
53.6.2 Operaciones requeridas	127
53.6.3 Reglas	127
53.7 4.1 ¿Por qué utilizar <code>do...while</code> ?	127
53.8 Diseño previo	128
53.9 Diagrama de flujo	129
53.10 Definición de las opciones con <code>enum</code>	130
53.11 Construcción paso a paso	130
53.11.1 Paso 1: variables principales	130
53.11.2 Paso 2: mostrar el menú	130
53.11.3 Paso 3: procesar la opción	130
53.11.4 Paso 4: repetir el menú	131
53.12 Programa completo	131
53.13 Pruebas del programa	133
53.14 Reto opcional	133
53.15 Cierre	133
54 Capítulo 5. Integración de conceptos: construyendo un programa completo	135
55 Introducción	137
56 Objetivos	139
57 Caso de estudio	141
57.1 Sistema de Control de Parqueos	141
58 Análisis del problema	143
59 Variables	145
60 Pseudocódigo	147
61 Diagrama: Sistema de control de parqueos	149
62 Paso 1	151

63 Paso 2	153
64 Paso 3. Entrada	155
65 Paso 4. Tarifa	157
66 Paso 5	159
67 Paso 6	161
68 Reflexión	163
69 Resumen	165
70 Paso 7. Mostrando los resultados	167
70.1 Ejemplo de ejecución	168
70.2 Reflexionemos	168
71 Paso 8. Analizando el programa	169
72 Paso 8. ¿Cómo podemos mejorarlo?	171
72.1 Lo que construiremos a continuación	171
72.2 Preparándonos para la siguiente clase	171
73 Capítulo 6. Organizando nuestros programas mediante funciones y procedimientos	173
73.1 Caso de estudio	173
73.1.1 Evolucionando el Sistema de Control de Parqueos	173
74 Introducción	175
75 Objetivos	177
76 ¿Por qué necesitamos organizar nuestros programas?	179
77 La analogía de la construcción de una casa	181
78 Procedimientos y funciones	183
79 ¿Qué es un procedimiento?	185
80 ¿Qué es una función?	187
81 Procedimiento o función	189
82 ¿Qué son los parámetros?	191
83 La analogía de la casa	193
84 Transformando nuestro programa	195
85 Primera mejora: mover el título del programa	197
85.1 Objetivo	197

86 Segunda mejora: separar la lectura de datos	199
86.1 Objetivo	199
87 Tercera mejora: crear una función para determinar la tarifa	201
87.1 Objetivo	201
88 Cuarta mejora: crear una función para calcular el total	203
89 Quinta mejora: crear un procedimiento para mostrar el reporte	205
90 Nuestro nuevo programa	207
91 ¿Qué ganamos con esta organización?	209
92 Primera modificación: crear el procedimiento <code>mostrarTitulo()</code>	211
93 1. Identificamos el código que vamos a mover	213
94 2. Creamos el procedimiento	215
95 3. Modificamos <code>main()</code>	217
96 4. Código después de la primera modificación	219
97 Segunda modificación: crear el procedimiento <code>leerDatos()</code>	223
98 1. Identificamos el código que vamos a mover	225
99 2. Creamos el procedimiento	227
1003. Modificamos <code>main()</code>	229
1014. ¿Qué ganamos?	231
101.1Antes	231
101.2Después	231
101.3Preparándonos para la siguiente modificación	232
102Tercera modificación: crear la función <code>determinarTarifa()</code>	233
1031. Identificamos el código que vamos a mover	235
1042. Creamos la función	237
104.1Analicemos el encabezado	237
1053. Modificamos <code>main()</code>	239
1064. ¿Qué sucede internamente?	241
1075. ¿Qué ganamos?	243
107.1Antes	243
107.2Después	243
107.3Preparándonos para la siguiente modificación	243

108Cuarta modificación: crear la función <code>calcularTotal()</code>	245
1091. Identificamos el código que vamos a mover	247
1102. Creamos la función	249
1113. Modificamos <code>main()</code>	251
1124. ¿Qué sucede internamente?	253
1135. ¿Qué ganamos?	255
113.1Antes	255
113.2Después	255
113.3Preparándonos para la siguiente modificación	256
114Quinta modificación: crear el procedimiento <code>mostrarResultados()</code>	257
1151. Identificamos el código que vamos a mover	259
1162. Creamos el procedimiento	261
1173. Modificamos <code>main()</code>	263
1184. Nuestro nuevo <code>main()</code>	265
1195. ¿Qué cambió?	267
119.1Antes	267
119.2Después	267
120Comprobación final	269
121Lo que aprendimos	271
122Preparándonos para el siguiente capítulo	273
123Cierre del capítulo	275
123.1Próximo paso	275

1 Créditos Editoriales

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas

ALGORITMOS

Introducción y Fundamentos

Editorial Académica

Ingeniería en Sistemas

Ing. Jorge Luis García Chacón

Segundo Semestre

Edición 2026

Diseñar una vez.

Enseñar toda la vida.

2 Acerca de este volumen

Este libro forma parte de la colección **Editorial Académica de Ingeniería en Sistemas**.

Su propósito es proporcionar material de estudio moderno, visual y práctico para apoyar el proceso de enseñanza y aprendizaje.

Cada capítulo ha sido diseñado para combinar teoría, ejemplos, diagramas, laboratorios y ejercicios que permitan al estudiante comprender los conceptos y aplicarlos en situaciones reales.

3 Presentación del curso

Bienvenidos al curso de **Algoritmos** para estudiantes de Ingeniería en Sistemas.

Este documento inicia una ruta de aprendizaje que busca que el estudiante no solo aprenda instrucciones, símbolos o código, sino que desarrolle una forma ordenada de pensar para resolver problemas.

Idea central del curso

Al finalizar el curso, el estudiante debe ser capaz de plantear una solución a un problema mediante un algoritmo y, posteriormente, expresarlo como un programa ejecutado por un computador.

3.1 Objetivo general de la asignatura

Resolver problemas básicos a través de la construcción de programas basados en algoritmos y un lenguaje de programación, generando acciones hacia la búsqueda de propuestas pertinentes.

3.2 Aprendizajes esperados

Al finalizar la asignatura, el estudiante será capaz de:

- Descomponer un problema en subproblemas para resolverlo de forma más sencilla.
- Interpretar algoritmos y programas para resolver problemas básicos de programación.
- Construir algoritmos y programas aplicados a problemas matemáticos sencillos.
- Trabajar colaborativamente, cumpliendo un rol y responsabilizándose de él.
- Comprender situaciones problemáticas que tienen un componente informático.

Consejo del Ingeniero

Antes de escribir código, primero piense cómo resolvería el problema con lápiz y papel. Programar es más fácil cuando el algoritmo ya está claro.

4 ¿Qué es un algoritmo?

Un **algoritmo** es un procedimiento detallado para resolver un problema mediante una serie de pasos ordenados, precisos y finitos.

En términos sencillos:

Un algoritmo indica qué hacer, en qué orden hacerlo y cuándo terminar.

4.1 Elementos básicos de un algoritmo

Todo algoritmo puede analizarse con la siguiente estructura:

Elemento	Pregunta que responde	Ejemplo
Entrada	¿Qué datos necesito?	Dos números
Proceso	¿Qué operaciones realizo?	Sumar los números
Salida	¿Qué resultado entrego?	Resultado de la suma

! Importante

Un algoritmo debe llevarnos desde un **estado inicial** hasta un **estado final**. Si no sabemos dónde empieza o dónde termina, el algoritmo estará incompleto.

4.2 Ejemplo cotidiano

Pensemos en preparar una taza de café.

Entrada

- Agua
- Café
- Azúcar
- Taza

Proceso

1. Calentar el agua.

2. Colocar café en la taza.
3. Agregar agua caliente.
4. Agregar azúcar si se desea.
5. Mezclar.

Salida

- Una taza de café lista para tomar.

Aunque este ejemplo parece simple, ya contiene los tres elementos fundamentales: entrada, proceso y salida.

5 ¿Cómo desarrollar un algoritmo?

Para desarrollar algoritmos se necesita práctica, imaginación y orden.

Algunas ideas clave son:

- **Imaginación:** buscar una forma de resolver el problema.
- **No reinventar la rueda:** aprovechar soluciones conocidas cuando sea posible.
- **Dividir para conquistar:** partir un problema grande en problemas más pequeños.
- **Practicar constantemente:** la lógica se fortalece con ejercicios.

Consejo del Ingeniero

Cuando un problema parezca difícil, no intente resolverlo completo de una vez. Divídalo en partes pequeñas y resuelva una por una.

6 ¿Cómo se describe un algoritmo?

Un algoritmo puede describirse de varias formas.

Forma	Descripción	Uso principal
Lenguaje natural	Explicación con palabras comunes	Primer análisis del problema
Pseudocódigo	Instrucciones ordenadas parecidas a código	Diseño del algoritmo
Diagrama de flujo	Representación gráfica del algoritmo	Visualizar decisiones y procesos
Lenguaje de programación	Código ejecutable por la computadora	Implementación final

Precisión

Un algoritmo no puede ser ambiguo. Todos deben entender lo mismo: el docente, los estudiantes y, finalmente, el computador.

7 Variables: de neuronas a espacios de memoria

Para entender las variables, imaginemos cómo resolveríamos mentalmente una suma de dos números.

Primero pedimos el primer valor y lo recordamos. Luego pedimos el segundo valor y también lo recordamos. Después sumamos ambos valores y guardamos el resultado.

En nuestro cerebro podríamos imaginar tres espacios:

- Espacio 1: primer número.
- Espacio 2: segundo número.
- Espacio 3: resultado.

En programación, esos espacios se llaman **variables**.

Definición

Una variable es un espacio de memoria que permite almacenar un dato para utilizarlo durante la ejecución de un algoritmo o programa.

7.1 Algoritmo para sumar dos números

Problema: Crear un algoritmo que sume dos números ingresados por el usuario.

Entrada

- Primer número.
- Segundo número.

Proceso

- Sumar ambos números.

Salida

- Resultado de la suma.

7.2 Pseudocódigo

```
Algoritmo SumarDosNumeros
Variables
    Entero var1, var2, var3
Inicio
    Escribir "Ingrese el primer valor"
    Leer var1

    Escribir "Ingrese el segundo valor"
    Leer var2

    var3 = var1 + var2

    Escribir "El resultado es: ", var3
Fin
```

7.3 Explicación paso a paso

Paso	Acción	Resultado
1	Definir variables	var1, var2, var3 existen en memoria
2	Leer primer valor	var1 almacena el primer número
3	Leer segundo valor	var2 almacena el segundo número
4	Sumar	var3 almacena var1 + var2
5	Mostrar resultado	Se imprime var3

8 Ejercicio guiado: multiplicar tres números

Problema: Crear un algoritmo que multiplique tres números ingresados por teclado.

8.1 Análisis

Entrada

- Primer número.
- Segundo número.
- Tercer número.

Proceso

- Multiplicar los tres valores.

Salida

- Resultado de la multiplicación.

8.2 Pseudocódigo

```
Algoritmo MultiplicarTresNumeros
Variables
    Entero var1, var2, var3, var4
Inicio
    Escribir "Ingresa el primer valor"
    Leer var1

    Escribir "Ingresa el segundo valor"
    Leer var2

    Escribir "Ingresa el tercer valor"
    Leer var3
```

```
var4 = var1 * var2 * var3  
Escribir "El resultado es: ", var4  
Fin
```

 Observe

Para multiplicar tres números necesitamos cuatro variables: tres para guardar las entradas y una para guardar el resultado.

9 Diagramas de flujo

Un **diagrama de flujo** es una representación gráfica de la lógica de un algoritmo.

Permite observar el orden de las instrucciones, las entradas, los procesos, las salidas y las decisiones.

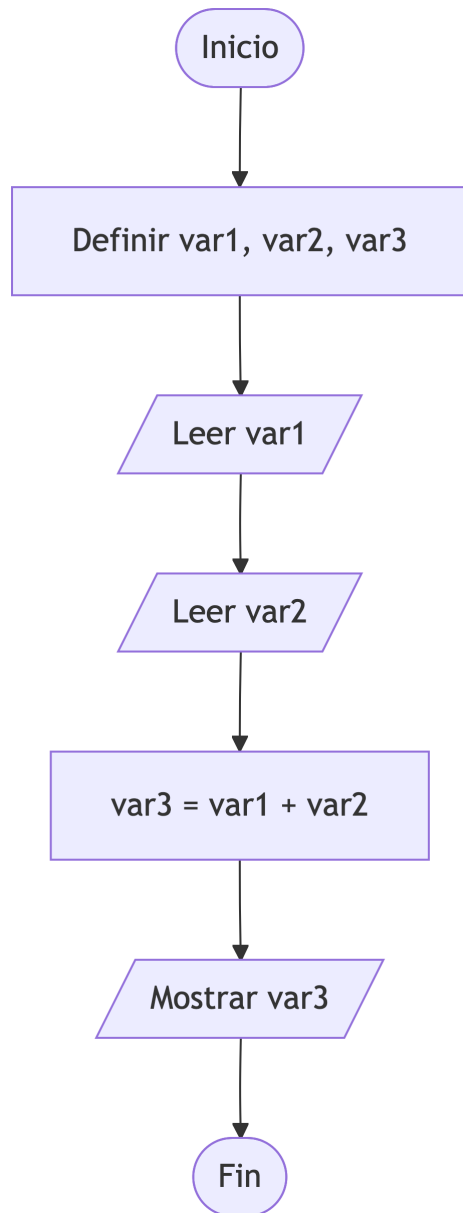
9.1 Nomenclatura básica

Símbolo	Nombre	Significado
□	Óvalo	Inicio o fin del algoritmo
□	Paralelogramo	Entrada o salida de datos
□	Rectángulo	Proceso u operación
□	Rombo	Decisión o bifurcación
↓ / →	Flechas	Dirección del flujo
•	Círculo pequeño	Conector

i Nota para el docente

En el PDF se utilizan símbolos simples para que el documento sea liviano y fácil de renderizar. En clase, estos mismos símbolos se explican con dibujos más grandes en la pizarra o en diagramas de flujo.

9.2 Ejemplo gráfico del flujo para sumar dos números



9.3 Ejemplo textual del flujo para sumar dos números

```
Inicio
↓
Definir var1, var2, var3
↓
Leer var1
```

```
↓  
Leer var2  
↓  
var3 = var1 + var2  
↓  
Mostrar var3  
↓  
Fin
```

i Para la clase

Primero puede mostrarse el diagrama gráfico y luego el flujo textual. La idea es que el estudiante relacione cada símbolo con una acción concreta del algoritmo.

10 Bifurcaciones y decisiones

Una **bifurcación** permite que el algoritmo tome caminos diferentes según una condición.

Las condiciones siempre se evalúan como:

- Verdadero.
- Falso.

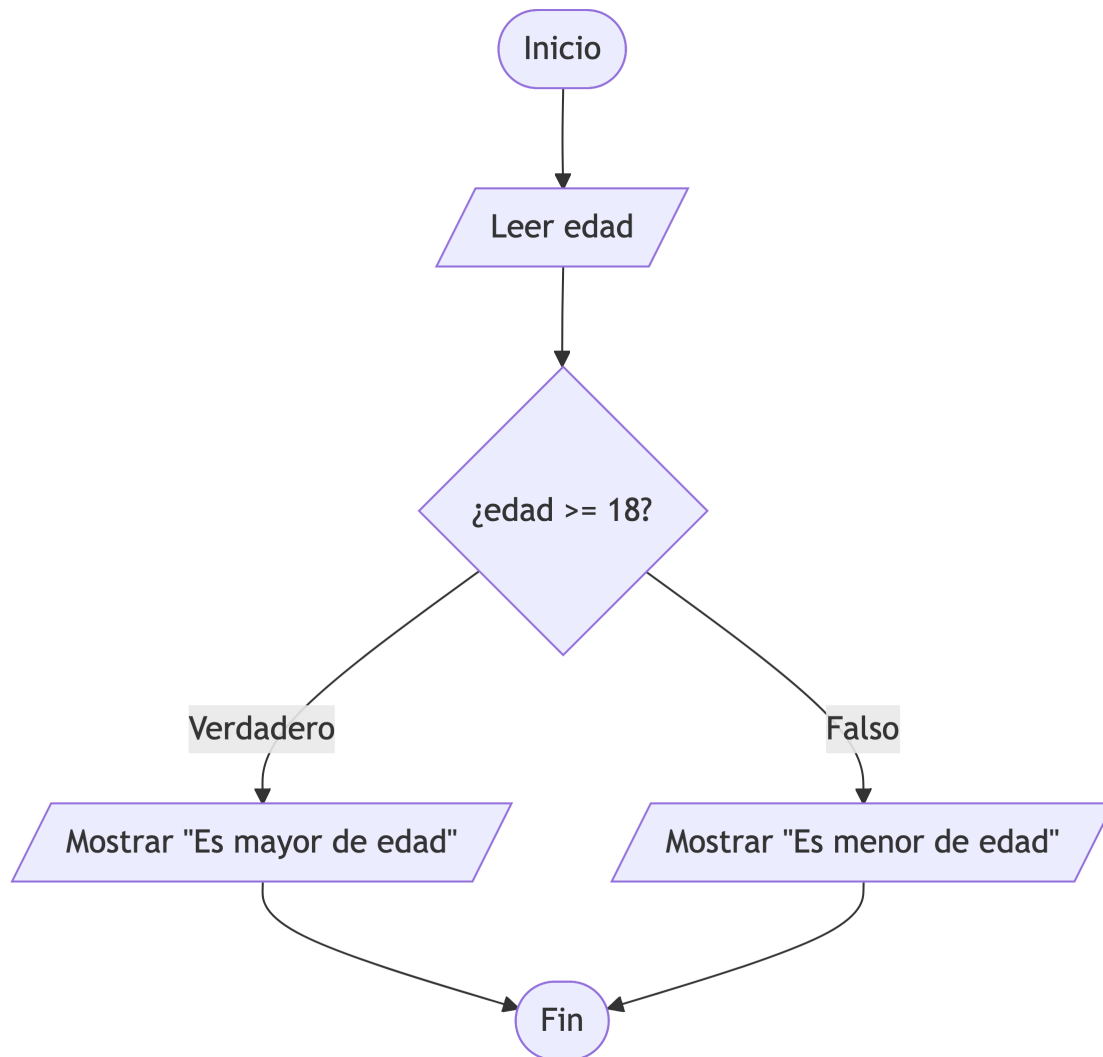
10.1 Ejemplo: mayor de edad

Problema: Crear un algoritmo donde una persona ingrese su edad y se muestre si es mayor de edad.

```
Algoritmo MayorDeEdad
Variables
    Entero edad
Inicio
    Escribir "Ingrese su edad"
    Leer edad

    Si edad >= 18 Entonces
        Escribir "Es mayor de edad"
    Sino
        Escribir "Es menor de edad"
    FinSi
Fin
```

10.2 Diagrama de flujo



10.3 Flujo lógico

```
Inicio
↓
Leer edad
↓
¿edad >= 18?
├── Verdadero → Mostrar "Es mayor de edad"
└── Falso     → Mostrar "Es menor de edad"
↓
Fin
```

! Idea clave

En las bifurcaciones, el algoritmo no ejecuta todo. Ejecuta solamente el camino que corresponde al resultado de la condición.

11 Operadores

Las condiciones se construyen utilizando operadores.

En esta primera clase revisaremos tres grupos:

- Operadores aritméticos.
- Operadores relacionales.
- Operadores lógicos.

12 Operadores aritméticos

Operador	Significado	Ejemplo	Resultado
*	Multiplicación	5 * 2	10
/	División	10 / 2	5
+	Suma	4 + 3	7
-	Resta	8 - 5	3
%	Módulo o residuo	5 % 2	1

12.1 Módulo o residuo

El operador % devuelve el residuo de una división entera.

```
5 % 2 = 1
4 % 2 = 0
```

Uso frecuente

El operador módulo se utiliza mucho para saber si un número es par o impar.
Si `numero % 2 == 0`, entonces el número es par.

13 Operadores relacionales

Los operadores relacionales comparan valores y devuelven un resultado lógico: verdadero o falso.

Operador	Significado	Ejemplo
>	Mayor que	edad > 18
<	Menor que	edad < 18
>=	Mayor o igual que	edad >= 18
<=	Menor o igual que	nota <= 60
==	Igual que	a == b
!=	Diferente de	a != b

Error común

No es lo mismo = que ==.

- = se utiliza para asignar.
- == se utiliza para comparar.

14 Operadores lógicos

Los operadores lógicos permiten combinar condiciones.

Operador	Nombre	Significado práctico
& &	Y / AND	Exige que ambas condiciones sean verdaderas
	O / OR	Basta con que una condición sea verdadera
!	NO / NOT	Invierte el valor lógico

14.1 Operador Y: &&

El operador && es verdadero solamente cuando ambas condiciones son verdaderas.

A	B	A && B
V	V	V
V	F	F
F	V	F
F	F	F

Forma de recordarlo

El operador && es **exigente**: pide que todo se cumpla.

14.2 Operador O: | |

El operador | | es verdadero cuando al menos una condición es verdadera.

A	B	A B
V	V	V
V	F	V
F	V	V
F	F	F

i Forma de recordarlo

El operador $||$ es **flexible**: acepta que se cumpla una condición o la otra.

14.3 Operador NO: !

El operador $!$ invierte el valor lógico.

A	$!A$
V	F
F	V

i Forma de recordarlo

El operador $!$ voltea la respuesta: lo verdadero se vuelve falso y lo falso se vuelve verdadero.

15 Casos de análisis

A continuación se presentan casos para practicar el análisis de expresiones lógicas.

La idea no es memorizar la respuesta, sino resolver paso a paso.

16 Caso 1: operador &&

Datos iniciales:

```
var1 = 2  
var2 = 0  
var3 = 4
```

Operación:

```
var4 = (var1 * var1) / var3
```

Sustituyendo valores:

```
var4 = (2 * 2) / 4  
var4 = 4 / 4  
var4 = 1
```

Condición:

```
var4 >= 1 && var1 != 0
```

Análisis:

```
var4 >= 1 → 1 >= 1 → Verdadero  
var1 != 0 → 2 != 0 → Verdadero
```

Resultado final:

```
Verdadero && Verdadero = Verdadero
```

Conclusión

En este caso, el resultado de la condición es **Verdadero**.

17 Caso 2: combinación de || y &&

Datos iniciales:

```
var1 = 2  
var2 = 0  
var3 = 4
```

Operación:

```
var4 = var3 % var1
```

Sustituyendo valores:

```
var4 = 4 % 2  
var4 = 0
```

Condición:

```
(var4 != 0 || var2 > 0) && (var3 == 4)
```

Análisis:

```
var4 != 0 → 0 != 0 → Falso  
var2 > 0 → 0 > 0 → Falso
```

Primera parte:

```
Falso || Falso = Falso
```

Segunda parte:

```
var3 == 4 → 4 == 4 → Verdadero
```

Resultado final:

Falso && Verdadero = Falso

 Conclusión

En este caso, el resultado de la condición es **Falso**.

18 Caso 3: operador !

Datos iniciales:

```
var1 = 2  
var4 = 1  
var3 = 4
```

Operación:

```
var4 = var4 + var1
```

Sustituyendo valores:

```
var4 = 1 + 2  
var4 = 3
```

Condición original:

```
!(var1 && (var4 >= var1))
```

Para estudiantes que están iniciando, conviene escribirla de forma más explícita:

```
!(var1 != 0 && (var4 >= var1))
```

Análisis:

```
var1 != 0      → 2 != 0 → Verdadero  
var4 >= var1   → 3 >= 2 → Verdadero
```

Primera parte:

```
Verdadero && Verdadero = Verdadero
```

Aplicando el operador NO:

```
!Verdadero = Falso
```

Conclusión

En este caso, el resultado de la condición es **Falso**.

Nota de C++

En C++:

- 0 se interpreta como falso.
- Cualquier valor distinto de 0 se interpreta como verdadero.

Por eso una expresión como `if (var1)` puede funcionar, aunque para iniciar es mejor escribir condiciones explícitas como `var1 != 0`.

19 Prioridad de operadores

Es importante notar que existen prioridades entre operadores.

Por ejemplo, no siempre se evalúa de izquierda a derecha sin reglas. Por eso, cuando una expresión tenga varias operaciones, es recomendable usar paréntesis.

! Recomendación

Use paréntesis cuando una condición combine varios operadores. Ayuda al computador y también ayuda a quien lee el algoritmo.

Ejemplo más claro:

```
(var4 != 0 || var2 > 0) && (var3 == 4)
```

Ejemplo menos claro:

```
var4 != 0 || var2 > 0 && var3 == 4
```


20 Introducción a ciclos

Un **ciclo** es la repetición de un conjunto de instrucciones.

El ciclo termina cuando se cumple una condición de finalización.

20.1 ¿Cuándo se aplican los ciclos?

Los ciclos se aplican cuando queremos ejecutar un conjunto de instrucciones varias veces.

Por ejemplo:

- Sumar varios números.
- Pedir datos de varios estudiantes.
- Buscar el número mayor de una lista.
- Repetir un menú hasta que el usuario decida salir.

i Idea clave

Si una acción debe repetirse, probablemente necesitamos un ciclo.

21 Ejemplo: sumar N números

Problema: Crear un algoritmo que sume N números ingresados por teclado.

21.1 Análisis

Entrada

- Cantidad de números a sumar: N .
- Cada uno de los números.

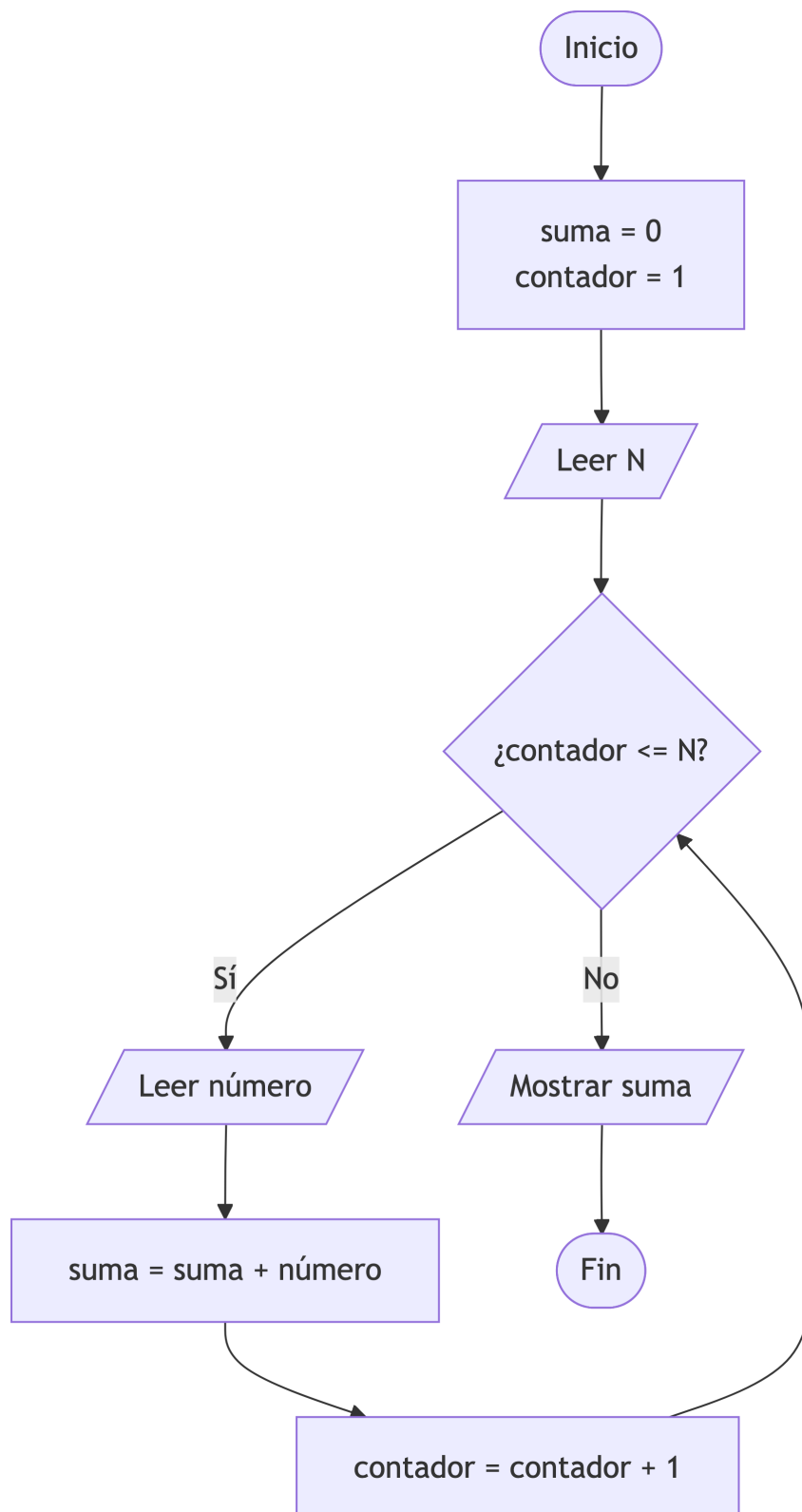
Proceso

- Repetir la lectura de números.
- Acumular la suma.

Salida

- Total de la suma.

21.2 Diagrama de flujo



21.3 Pseudocódigo

```
Algoritmo SumarNNumeros
Variables
    Entero N, contador, numero, suma
Inicio
    suma = 0
    contador = 1

    Escribir "Ingrese cuántos números desea sumar"
    Leer N

    Mientras contador <= N Hacer
        Escribir "Ingrese un número"
        Leer numero
        suma = suma + numero
        contador = contador + 1
    FinMientras

    Escribir "La suma total es: ", suma
Fin
```


22 Ejemplo en C++: sumar del 1 hasta N

```
#include <iostream>
using namespace std;

int main() {
    int N, suma = 0, cont = 1;

    cout << "Ingrese hasta qué número desea sumar: ";
    cin >> N;

    while (cont <= N) {
        suma += cont;
        cont++;
    }

    cout << "El resultado es: " << suma << endl;
    return 0;
}
```

i Nota

En C++, endl permite hacer un salto de línea. También puede usarse "\n".

23 Ejemplo en C++: encontrar el mayor de N números

```
#include <iostream>
using namespace std;

int main() {
    int N, var, mayor, cont = 1;

    cout << "Ingrese cuántos números va a ingresar: ";
    cin >> N;

    cout << "Ingrese el valor 1: ";
    cin >> var;
    mayor = var;

    while (cont < N) {
        cout << "Ingrese el valor " << cont + 1 << ": ";
        cin >> var;

        if (var > mayor) {
            mayor = var;
        }

        cont++;
    }

    cout << "El número mayor es: " << mayor << endl;

    return 0;
}
```

Observe

El primer número se guarda inicialmente como `mayor`. Luego, cada nuevo número se compara contra ese valor.

24 Programa de aplicación: ecuación de primer grado

Resolver una ecuación de primer grado de la forma:

$$a x + b = 0$$

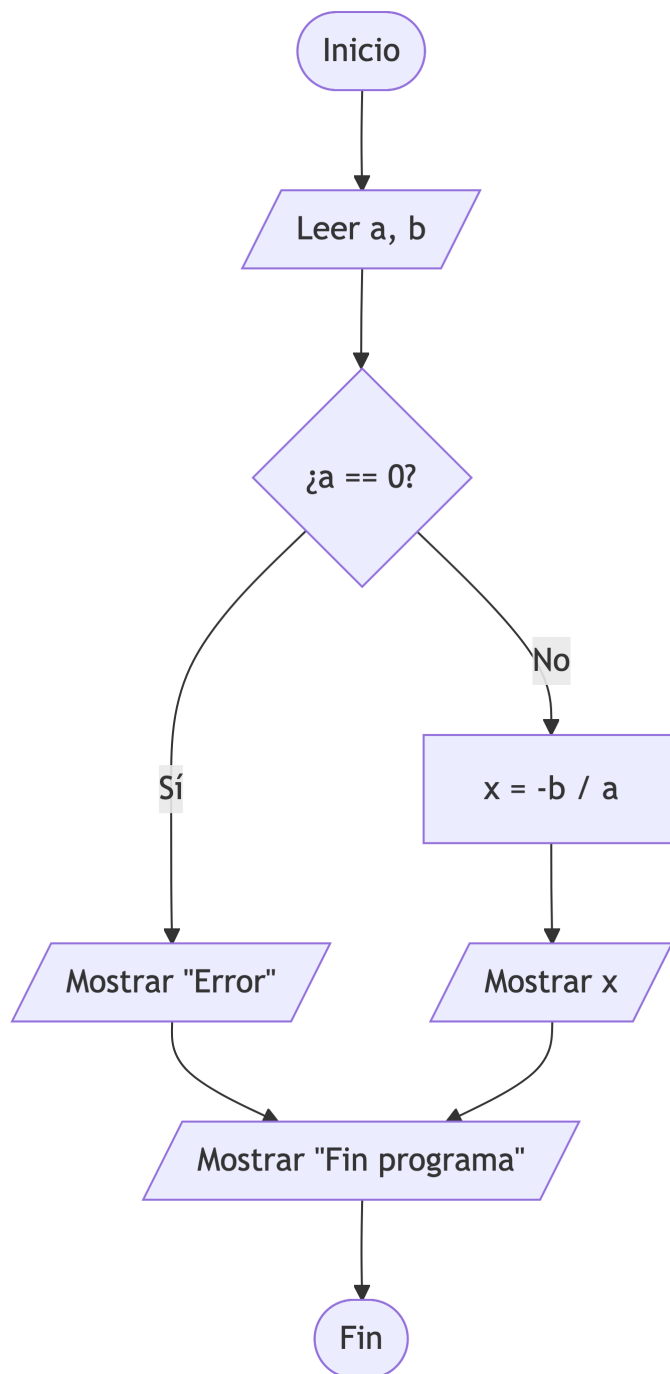
La solución es:

$$x = -b / a$$

Pero existe un caso especial:

Si $a = 0$, la división no puede realizarse.

24.1 Diagrama de flujo



24.2 Pseudocódigo

```
Algoritmo EcuacionPrimerGrado
Variables
    Entero a, b, x
Inicio
    Escribir "Ingrese valores de a y b"
    Leer a, b

    Si a == 0 Entonces
        Escribir "Error: no se puede dividir entre cero"
    Sino
        x = -b / a
        Escribir "La solución es: ", x
    FinSi

    Escribir "Fin del programa"
Fin
```

24.3 Código en C++

```
#include <iostream>
using namespace std;

int main() {
    int a, b, x;

    cout << "Ingrese valores de EC (a y b): ";
    cin >> a >> b;

    if (a == 0) {
        cout << "Error" << endl;
    } else {
        x = -b / a;
        cout << "La solución es: " << x << endl;
    }

    cout << "Fin programa" << endl;
    return 0;
}
```

Observación importante

En este ejemplo `x` está declarada como entero. Más adelante puede mejorarse usando variables reales para obtener resultados decimales.

25 Ejercicios para resolver en clase

25.1 Ejercicio 1

Cree un algoritmo que multiplique tres números ingresados por teclado y muestre el resultado.

25.2 Ejercicio 2

Cree un algoritmo que divida dos números ingresados por teclado y muestre el resultado.

Tome en cuenta que no se debe dividir entre cero.

25.3 Ejercicio 3

Cree un algoritmo que permita sumar tres números ingresados por teclado y muestre el resultado.

25.4 Ejercicio 4

Cree un algoritmo donde una persona ingrese su edad y muestre si es mayor de edad.

25.5 Ejercicio 5

Cree un algoritmo que muestre en forma ordenada tres números enteros ingresados desde teclado, de mayor a menor.

25.6 Ejercicio 6

Cree un algoritmo que permita sumar N números y muestre el resultado. El valor de N debe ser ingresado por teclado, al igual que los números que se sumarán.

26 Resumen de la clase

En esta clase aprendimos que:

- Un algoritmo es una secuencia finita, ordenada y precisa de pasos.
- Todo algoritmo trabaja con entradas, procesos y salidas.
- Las variables permiten almacenar datos en memoria.
- El pseudocódigo permite expresar soluciones antes de programarlas.
- Los diagramas de flujo ayudan a visualizar la lógica del algoritmo.
- Las bifurcaciones permiten tomar decisiones.
- Los operadores aritméticos, relacionales y lógicos permiten construir condiciones.
- Los ciclos permiten repetir instrucciones.

Consejo final del Ingeniero

Un buen programador no es quien escribe código más rápido, sino quien piensa mejor la solución antes de programarla.

27 Tarea sugerida

Para reforzar la clase, resuelva los siguientes problemas en pseudocódigo y diagrama de flujo:

1. Calcular el área de un rectángulo.
2. Calcular el promedio de tres notas.
3. Determinar si un número es par o impar.
4. Determinar el mayor de dos números.
5. Sumar los números del 1 hasta N.

28 Cierre

Algoritmos – De la lógica a la programación

Material preparado para el curso de Algoritmos, Ingeniería en Sistemas, Universidad Mariano Gálvez.

Docente: Ing. Jorge Luis García Chacón

29 Clase 2. Primeros pasos en C++

Laboratorio No. 1

29.1 Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Utilizar OnlineGDB para escribir y ejecutar programas.
- Comprender la estructura básica de un programa en C++.
- Declarar variables.
- Leer datos desde el teclado.
- Mostrar resultados en pantalla.

29.2 Herramienta de trabajo

Trabajaremos utilizando:

https://www.onlinegdb.com/online_c++_compiler

29.3 Nuestro primer programa

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hola Ingeniería en Sistemas" << endl;
    return 0;
}
```

29.3.1 Explicación

Línea	Descripción
<code>#include <iostream></code>	Biblioteca para entrada y salida.
<code>using namespace std;</code>	Permite utilizar <code>cout</code> y <code>cin</code> sin escribir <code>std::</code> .
<code>int main()</code>	Punto de inicio del programa.
<code>{ }</code>	Delimitan el bloque principal.
<code>return 0;</code>	Finaliza correctamente el programa.

29.4 Entrada y salida

```
#include <iostream>
using namespace std;

int main()
{
    int edad;

    cout << "Ingrese su edad: ";
    cin >> edad;

    cout << "Usted tiene " << edad << " años." << endl;

    return 0;
}
```

29.5 Tipos de datos

Tipo	Ejemplo
int	25
float	25.5
double	3.14159265
char	'A'
string	"Guatemala"
bool	true

29.6 Ejercicio guiado

Crear un programa que solicite:

- Nombre
- Edad
- Carrera

y muestre la información ingresada.

29.7 Laboratorio

29.7.1 Ejercicio 1

Solicite dos números enteros y muestre su suma.

29.7.2 Ejercicio 2

Solicite tres números y muestre su multiplicación.

29.7.3 Ejercicio 3

Calcule el área de un rectángulo.

29.7.4 Ejercicio 4

Calcule el promedio de tres notas.

29.8 Introducción a while

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;

    while(i <= 5)
    {
        cout << i << endl;
        i++;
    }

    return 0;
}
```

Observe: Si elimina `i++`; el ciclo nunca finalizará.

29.9 Resumen

En esta clase aprendimos:

- La estructura de un programa en C++.
- Uso de `cout` y `cin`.
- Declaración de variables.
- Tipos de datos.
- Primer ciclo `while`.

29.10 Tarea

1. Programa que calcule el perímetro de un rectángulo.
2. Programa que calcule el promedio de cuatro notas.
3. Programa que determine si un número es positivo o negativo.
4. Programa que convierta grados Celsius a Fahrenheit.
5. Modifique el ejemplo del ciclo `while` para mostrar los números del 1 al 10.

30 Laboratorio 1. Desarrollo de una solución informática

Análisis, diseño e implementación de algoritmos

31 Laboratorio 1

31.1 Desarrollo de una solución informática

31.1.1 Objetivo

Aplicar los conocimientos adquiridos durante las primeras clases del curso mediante el análisis, diseño e implementación de una solución informática utilizando pseudocódigo, diagrama de flujo y el lenguaje de programación C++.

! Competencias que se evaluarán

Al finalizar este laboratorio el estudiante será capaz de:

- Analizar un requerimiento.
- Identificar entradas, procesos y salidas.
- Diseñar un algoritmo mediante pseudocódigo.
- Representar la solución utilizando un diagrama de flujo.
- Implementar el algoritmo en C++.
- Verificar el funcionamiento de la solución mediante casos de prueba.

32 Requerimiento

Una academia de capacitación desea automatizar el cálculo del promedio de sus estudiantes.

Para ello necesita un programa que solicite la siguiente información:

- Nombre del estudiante.
- Primera nota.
- Segunda nota.
- Tercera nota.

Posteriormente el sistema deberá:

1. Calcular el promedio de las tres notas.
2. Mostrar el promedio obtenido.
3. Indicar si el estudiante **Aprobó** o **Reprobó**.

Considere que:

- Un estudiante aprueba con un promedio mayor o igual a **61 puntos**.
 - En cualquier otro caso el estudiante reprueba.
-

33 Actividades a desarrollar

33.1 Parte 1. Análisis del requerimiento

Antes de escribir cualquier algoritmo complete la siguiente información.

33.1.1 Entradas

Escriba las entradas que necesita el sistema.

No.	Entrada
1	
2	
3	
4	

33.1.2 Procesos

Describa las operaciones que realizará el algoritmo.

No.	Proceso
1	
2	
3	

33.1.3 Salidas

Indique qué información mostrará el programa.

No.	Salida
1	
2	
3	

34 Parte 2. Diseño del algoritmo

34.1 Pseudocódigo

Diseñe el algoritmo utilizando pseudocódigo.

34.2 Diagrama de flujo

Diseñe el diagrama de flujo correspondiente.

35 Parte 3. Implementación

Desarrolle el programa utilizando el lenguaje **C++**.

El programa debe cumplir con todos los requerimientos indicados anteriormente.

Recuerde

Utilice únicamente los temas vistos en clase:

- Variables
- Entrada de datos (`cin`)
- Salida de datos (`cout`)
- Operadores aritméticos
- Operadores relacionales
- Estructuras de decisión (`if/else`)

No es necesario utilizar ciclos.

36 Parte 4. Casos de prueba

Ejecute el programa al menos dos veces utilizando diferentes datos.

36.1 Caso de prueba No. 1

Dato	Valor
Nombre	
Nota 1	
Nota 2	
Nota 3	
Promedio esperado	
Resultado esperado	

36.2 Caso de prueba No. 2

Dato	Valor
Nombre	
Nota 1	
Nota 2	
Nota 3	
Promedio esperado	
Resultado esperado	

37 Evidencia

Adjunte una captura de pantalla del programa en ejecución.

38 Criterios de evaluación

Criterio	Punteo
Identificación de entradas, procesos y salidas	15
Pseudocódigo	20
Diagrama de flujo	20
Programa en C++	35
Casos de prueba	10
Total	100

39 Conclusión

Describa brevemente qué aprendió durante el desarrollo del laboratorio.

Reflexión

En el desarrollo de software, escribir el código representa únicamente una parte del trabajo. Antes de programar, un desarrollador debe comprender el requerimiento, analizar la información disponible, diseñar una solución y posteriormente implementarla. Este laboratorio busca fortalecer esa forma de pensar, la cual será utilizada durante todo el curso.

40 Capítulo 3. Expresiones lógicas y estructuras condicionales

40.1 Presentación de la clase

En las clases anteriores aprendimos a identificar entradas, procesos y salidas; declarar variables; utilizar operadores aritméticos; trabajar con `DIV` y `MOD`; construir expresiones; y representar una solución mediante pseudocódigo, diagrama de flujo y código C++.

En esta clase daremos continuidad a esos temas incorporando las **expresiones lógicas** y las estructuras de decisión:

- `if`;
- `if...else`;
- `if...else if...else`;
- `switch...case`.

La meta no es memorizar sintaxis. La meta es aprender a decidir **qué estructura conviene usar según el problema**.

i Idea central

Una estructura condicional permite que el algoritmo tome caminos diferentes según el valor de una condición.

40.2 Distribución de los 80 minutos

Tiempo	Actividad
12 minutos	Introducción y repaso conceptual
18 minutos	Ejercicio guiado 1 con <code>if...else if...else</code>
18 minutos	El mismo ejercicio con <code>switch...case</code>
20 minutos	Ejercicio guiado 2: condiciones compuestas
7 minutos	Comparación, errores frecuentes y cierre
5 minutos	Explicación de la tarea

 Metodología de la clase

La explicación representa aproximadamente el 15 % de la sesión. El resto se dedica a analizar, diseñar, programar y probar soluciones.

41 Objetivos de aprendizaje

Al finalizar la clase, el estudiante será capaz de:

1. Evaluar expresiones lógicas.
2. Aplicar la precedencia de operadores.
3. Utilizar `if`, `else if` y `else`.
4. Utilizar `switch`, `case`, `break` y `default`.
5. Diferenciar cuándo conviene usar `if` y cuándo `switch`.
6. Elaborar el análisis de un problema.
7. Diseñar la solución mediante pseudocódigo y diagrama de flujo.
8. Implementar y probar la solución en C++.

42 Repaso rápido: expresiones y operadores

Una **expresión** es una combinación de variables, constantes y operadores que produce un valor.

```
int resultado = 3 + 4 * 2;
```

La multiplicación se evalúa antes que la suma:

```
3 + 4 * 2
3 + 8
11
```

42.1 Operadores aritméticos

Operador	Descripción	Ejemplo
+	Suma	10 + 3 = 13
-	Resta	10 - 3 = 7
*	Multiplicación	10 * 3 = 30
/	División	10 / 3 = 3 si ambas variables son enteras
%	Residuo	10 % 3 = 1

42.2 Operadores relacionales

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
==	Igual que
!=	Diferente de

42.3 Operadores lógicos

Operador	Nombre	Uso
& &	AND	Todas las condiciones deben ser verdaderas
	OR	Al menos una condición debe ser verdadera
!	NOT	Invierte el resultado lógico

42.4 Precedencia simplificada

Nivel	Operadores
1	()
2	!
3	*, /, %
4	+, -
5	<, >, <=, >=
6	==, !=
7	& &
8	
9	=

Consejo

Cuando una expresión sea larga, utilice paréntesis. Aunque el resultado sea correcto, los paréntesis hacen que la intención sea más clara.

43 Estructura `if...else if...else`

Se utiliza cuando necesitamos evaluar rangos, varias condiciones, operadores relacionales u operadores lógicos.

43.1 Sintaxis

```
if (condicion1) {  
    // instrucciones  
} else if (condicion2) {  
    // instrucciones  
} else {  
    // instrucciones  
}
```


44 Ejercicio guiado 1: calculadora con `if`

44.1 Requerimiento

Crear un programa que:

1. Solicite dos números.
2. Solicite una operación: +, -, *, / o %.
3. Realice la operación seleccionada.
4. Evite dividir entre cero.
5. Muestre un mensaje si la operación no es válida.

44.2 Parte A. Análisis

44.2.1 Entradas

Dato	Tipo
Primer número	Entero
Segundo número	Entero
Operación	Carácter

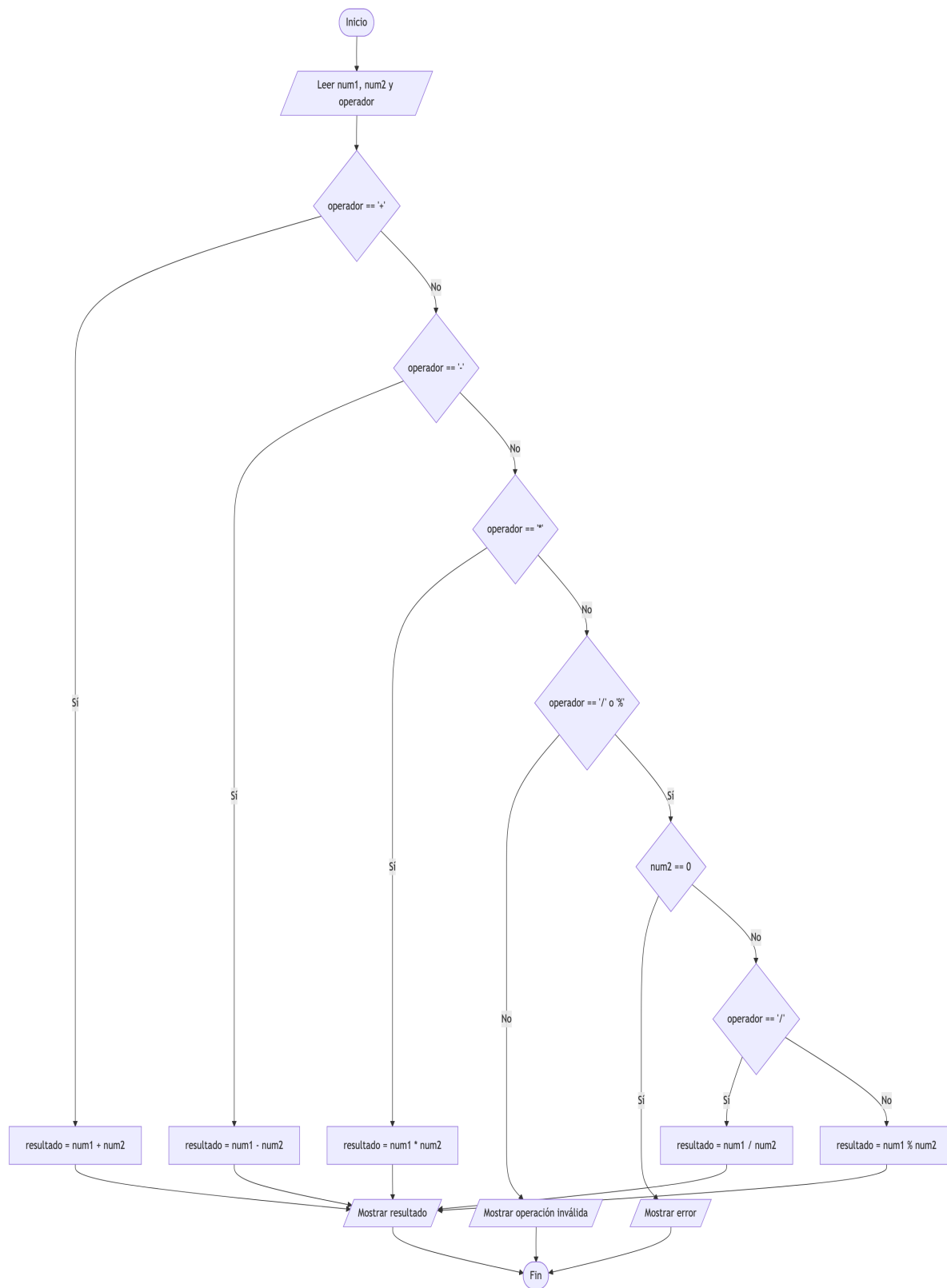
44.2.2 Procesos

1. Leer los dos números.
2. Leer la operación.
3. Comparar la operación ingresada.
4. Ejecutar el cálculo correspondiente.
5. Validar división y residuo entre cero.

44.2.3 Salidas

- Resultado de la operación.
- Mensaje de error cuando corresponda.

44.3 Parte B. Diagrama de flujo



44.4 Parte C. Pseudocódigo

```
Algoritmo CalculadoraConIf
Variables
    Entero num1, num2, resultado
    Caracter operador

Inicio
    Leer num1
    Leer num2
    Leer operador

    Si operador == '+' Entonces
        resultado = num1 + num2
        Escribir resultado
    Sino Si operador == '-' Entonces
        resultado = num1 - num2
        Escribir resultado
    Sino Si operador == '*' Entonces
        resultado = num1 * num2
        Escribir resultado
    Sino Si operador == '/' Entonces
        Si num2 == 0 Entonces
            Escribir "Error"
        Sino
            resultado = num1 / num2
            Escribir resultado
        FinSi
    Sino Si operador == '%' Entonces
        Si num2 == 0 Entonces
            Escribir "Error"
        Sino
            resultado = num1 % num2
            Escribir resultado
        FinSi
    Sino
        Escribir "Operación no válida"
    FinSi
Fin
```

44.5 Parte D. Código C++

```
#include <iostream>
using namespace std;
```

```
int main() {
    int num1, num2, resultado;
    char operador;

    cout << "Ingrese el primer numero: ";
    cin >> num1;

    cout << "Ingrese el segundo numero: ";
    cin >> num2;

    cout << "Ingrese la operacion (+, -, *, /, %): ";
    cin >> operador;

    if (operador == '+') {
        resultado = num1 + num2;
        cout << "Resultado: " << resultado << endl;
    } else if (operador == '-') {
        resultado = num1 - num2;
        cout << "Resultado: " << resultado << endl;
    } else if (operador == '*') {
        resultado = num1 * num2;
        cout << "Resultado: " << resultado << endl;
    } else if (operador == '/') {
        if (num2 == 0) {
            cout << "Error: no se puede dividir entre cero." << endl;
        } else {
            resultado = num1 / num2;
            cout << "Resultado: " << resultado << endl;
        }
    } else if (operador == '%') {
        if (num2 == 0) {
            cout << "Error: no se puede obtener residuo entre cero."
                << endl;
        } else {
            resultado = num1 % num2;
            cout << "Resultado: " << resultado << endl;
        }
    } else {
        cout << "Operacion no valida." << endl;
    }

    return 0;
}
```

}

45 El mismo ejercicio con `switch...case`

`switch` es apropiado cuando una misma variable se compara contra valores constantes exactos.

45.1 Sintaxis

```
switch (expresion) {  
    case valor1:  
        // instrucciones  
        break;  
  
    case valor2:  
        // instrucciones  
        break;  
  
    default:  
        // instrucciones  
}
```

45.2 Código C++

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int num1, num2, resultado;  
    char operador;  
  
    cout << "Ingrese el primer numero: ";  
    cin >> num1;  
  
    cout << "Ingrese el segundo numero: ";  
    cin >> num2;
```

```
cout << "Ingrese la operacion (+, -, *, /, %): ";
cin >> operador;

switch (operador) {
    case '+':
        resultado = num1 + num2;
        cout << "Resultado: " << resultado << endl;
        break;

    case '-':
        resultado = num1 - num2;
        cout << "Resultado: " << resultado << endl;
        break;

    case '*':
        resultado = num1 * num2;
        cout << "Resultado: " << resultado << endl;
        break;

    case '/':
        if (num2 == 0) {
            cout << "Error: no se puede dividir entre cero."
                << endl;
        } else {
            resultado = num1 / num2;
            cout << "Resultado: " << resultado << endl;
        }
        break;

    case '%':
        if (num2 == 0) {
            cout << "Error: no se puede obtener residuo entre ce
                << endl;
        } else {
            resultado = num1 % num2;
            cout << "Resultado: " << resultado << endl;
        }
        break;

    default:
        cout << "Operacion no valida." << endl;
}

return 0;
}
```

46 Comparación entre `if` y `switch`

Característica	<code>if...else if...else</code>	<code>switch...case</code>
Evalúa rangos	Sí	No directamente
Usa condiciones lógicas	Sí	No
Compara valores exactos	Sí	Sí
Claridad en menús	Buena	Muy buena
Uso recomendado	Rangos y condiciones complejas	Opciones exactas

! Regla práctica

Use `if` cuando la decisión dependa de rangos o expresiones lógicas. Use `switch` cuando una variable se compare contra opciones exactas.

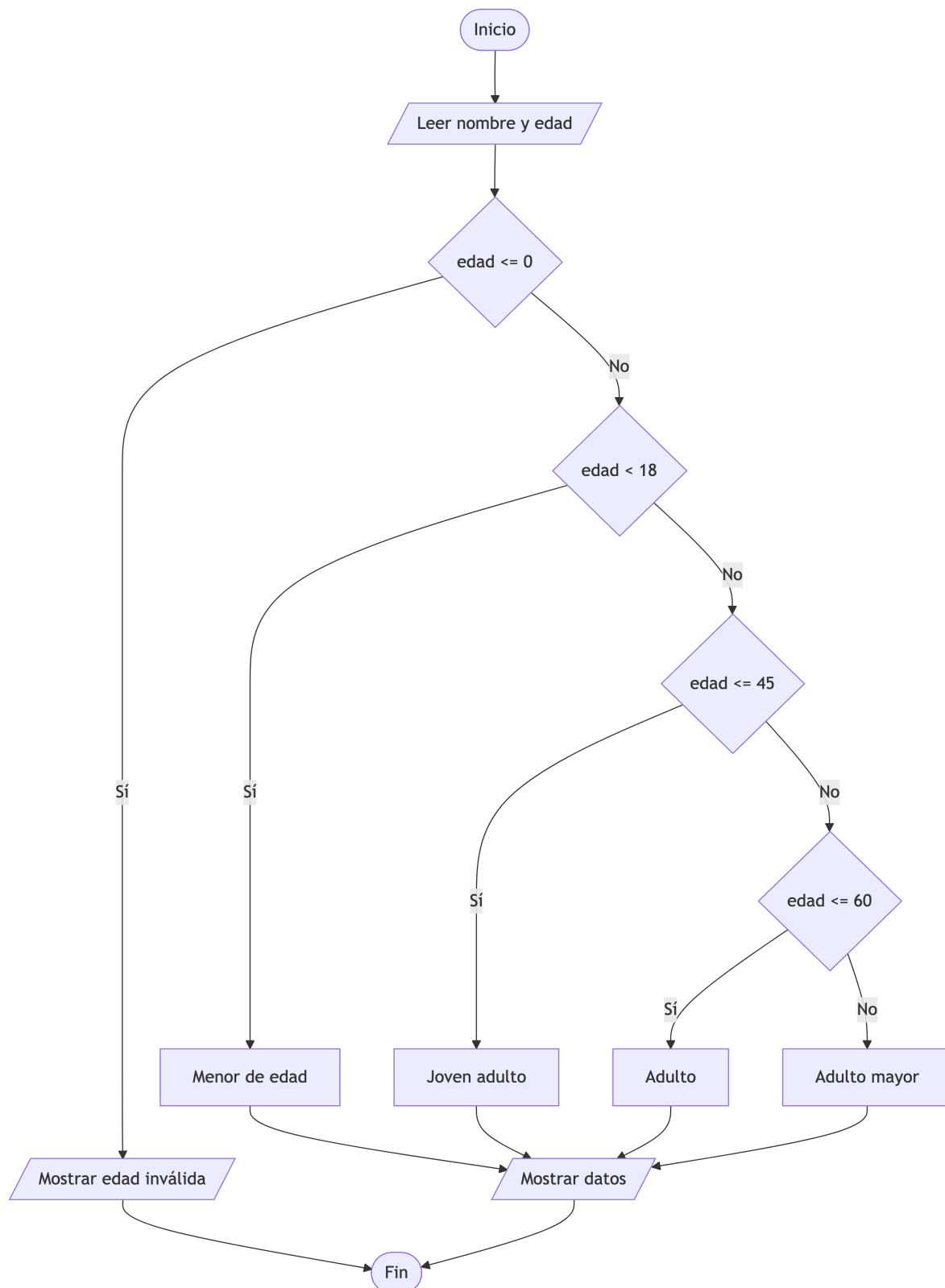
47 Ejercicio guiado 2: clasificación de edad

47.1 Requerimiento

Crear un programa que:

1. Solicite el nombre.
2. Solicite la edad.
3. No permita una edad igual o menor que cero.
4. Clasifique:
 - menor de edad: menos de 18;
 - joven adulto: de 18 a 45;
 - adulto: de 46 a 60;
 - adulto mayor: más de 60.
5. Muestre nombre, edad y clasificación.

47.2 Diagrama de flujo



47.3 Código C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombre;
    string clasificacion;
    int edad;

    cout << "Ingrese su nombre: ";
    getline(cin, nombre);

    cout << "Ingrese su edad: ";
    cin >> edad;

    if (edad <= 0) {
        cout << "Edad invalida." << endl;
    } else if (edad < 18) {
        clasificacion = "Menor de edad";
    } else if (edad <= 45) {
        clasificacion = "Joven adulto";
    } else if (edad <= 60) {
        clasificacion = "Adulto";
    } else {
        clasificacion = "Adulto mayor";
    }

    if (edad > 0) {
        cout << "\nNombre: " << nombre << endl;
        cout << "Edad: " << edad << endl;
        cout << "Clasificacion: " << clasificacion << endl;
    }

    return 0;
}
```


48 Actividad rápida en parejas

Modifique la calculadora para que también muestre el nombre de la operación seleccionada.

Cada pareja debe:

1. modificar la versión con `if`;
2. modificar la versión con `switch`;
3. probar al menos tres casos;
4. explicar cuál versión considera más clara.

49 Errores frecuentes

49.1 Usar = en lugar de ==

Incorrecto:

```
if (opcion = 1)
```

Correcto:

```
if (opcion == 1)
```

49.2 Colocar condición después de else

Incorrecto:

```
else (edad > 60)
```

Correcto:

```
else
```

49.3 Olvidar break

```
case 1:  
    cout << "Opcion 1";  
    break;
```

49.4 Usar switch para rangos

switch trabaja mejor con valores exactos.

50 Resumen de la clase

En esta clase aprendimos que:

- Las expresiones lógicas producen valores verdaderos o falsos.
- La precedencia determina el orden de evaluación.
- `if` permite evaluar rangos y condiciones complejas.
- `switch` facilita la construcción de menús y opciones exactas.
- El mismo problema puede resolverse con estructuras diferentes.
- Una buena solución comienza con análisis y diseño.
- Los casos de prueba permiten verificar el comportamiento del programa.

51 Tarea

51.1 Problema 1. Menú de conversión

Crear un programa que solicite una cantidad y muestre este menú:

```
1. Convertir metros a centímetros
2. Convertir centímetros a metros
3. Convertir kilómetros a metros
4. Salir
```

Debe resolverse utilizando `switch`.

51.1.1 Entregables

1. Análisis:
 - entradas;
 - procesos;
 - salidas.
2. Diagrama de flujo.
3. Pseudocódigo.
4. Código C++.
5. Tres casos de prueba.

51.2 Problema 2. Clasificación de notas

Solicitar una nota entre 0 y 100.

Clasificar:

Rango	Resultado
90 a 100	Excelente
80 a 89	Muy bueno
70 a 79	Bueno

Rango	Resultado
61 a 69	Aprobado
0 a 60	Reprobado

Debe resolverse utilizando `if...else if...else`.

51.2.1 Validación

Si la nota es menor que 0 o mayor que 100, mostrar:

```
Nota inválida
```

51.2.2 Entregables

1. Análisis:

- entradas;
- procesos;
- salidas.

2. Diagrama de flujo.

3. Pseudocódigo.

4. Código C++.

5. Cinco casos de prueba.

 Condición de entrega

No se calificará únicamente el código. La tarea debe incluir análisis, diagrama, pseudocódigo, programa y pruebas.

52 Cierre

Consejo final del Ingeniero

Primero comprenda la decisión que debe tomar el algoritmo. Después elija `if` o `switch`. La sintaxis viene al final.

53 Laboratorio No. 3: ciclos, enum y simulación de un cajero automático

53.1 Propósito del laboratorio

En este laboratorio repasaremos brevemente las estructuras que ya conocemos y construiremos un programa completo que combine decisiones, repetición y opciones de menú.

Al finalizar, el estudiante podrá:

- reconocer cuándo utilizar `if`, `switch`, `while`, `for` y `do...while`;
- construir ciclos `for` para repeticiones conocidas;
- representar opciones mediante un tipo enumerado `enum`;
- integrar un menú repetitivo con `switch` y `do...while`;
- validar depósitos y retiros en una simulación de cajero automático.

i Duración total

80 minutos. El propósito no es estudiar muchas estructuras nuevas, sino aprender a combinarlas en un programa claro y funcional.

53.2 Ruta de trabajo

Momento	Actividad	Tiempo
1	Recordatorio de estructuras básicas	10 min
2	Ciclo <code>for</code> : explicación y ejercicios	15 min
3	Uso de <code>enum</code>	10 min
4	Construcción del cajero automático	40 min
5	Prueba y cierre	5 min

53.3 Recordatorio rápido

Un programa puede imaginarse como una persona que sigue instrucciones:

- una **secuencia** indica qué hacer paso a paso;
- una **decisión** permite escoger entre varios caminos;
- un **ciclo** permite repetir una actividad;
- un **menú** reúne varias operaciones y espera la selección del usuario.

53.3.1 Estructura secuencial

Las instrucciones se ejecutan en el mismo orden en que aparecen.

```
#include <iostream>
using namespace std;

int main() {
    int numero1, numero2, suma;

    cout << "Ingrese el primer numero: ";
    cin >> numero1;

    cout << "Ingrese el segundo numero: ";
    cin >> numero2;

    suma = numero1 + numero2;

    cout << "La suma es: " << suma << endl;

    return 0;
}
```

Analogía: preparar café implica seguir una secuencia: calentar agua, agregar café, servir y endulzar.

53.3.2 Decisión con `if`

`if` se utiliza cuando una acción depende de una condición.

```
if (edad >= 18) {
    cout << "Es mayor de edad." << endl;
} else {
    cout << "Es menor de edad." << endl;
}
```

Analogía: antes de salir, una persona observa el clima. Si está lloviendo, lleva paraguas; de lo contrario, sale sin él.

53.3.3 Selección múltiple con `switch`

`switch` es útil cuando una variable puede representar varias opciones concretas.

```
switch (opcion) {  
    case 1:  
        cout << "Consultar saldo" << endl;  
        break;  
  
    case 2:  
        cout << "Depositar" << endl;  
        break;  
  
    case 3:  
        cout << "Retirar" << endl;  
        break;  
  
    default:  
        cout << "Opcion no valida" << endl;  
}
```

¿Para qué sirve `break`?

`break` detiene el `switch` después de ejecutar el caso seleccionado. Sin él, el programa puede continuar ejecutando los casos siguientes.

53.3.4 Repetición con `while`

`while` repite un bloque mientras una condición sea verdadera.

```
int contador = 1;  
  
while (contador <= 5) {  
    cout << contador << endl;  
    contador++;  
}
```

Analogía: mientras todavía haya platos sucios, se continúa lavando.

53.4 Ciclo for

El ciclo `for` se utiliza principalmente cuando conocemos cuántas veces debe repetirse una acción.

```
for (inicializacion; condicion; actualizacion) {  
    // instrucciones que se repiten  
}
```

En el siguiente ejemplo:

```
for (int i = 1; i <= 5; i++) {  
    cout << i << endl;  
}
```

ocurre lo siguiente:

1. `int i = 1` crea e inicializa el contador.
2. `i <= 5` determina si el ciclo continúa.
3. `i++` aumenta el contador después de cada repetición.

Analogía: recorrer cinco escritorios para entregar una hoja. Se conoce desde el inicio cuántos escritorios deben visitarse.

53.4.1 Ejercicio 1: mostrar los números del 1 al 10

Complete y ejecute el siguiente programa:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    for (int i = 1; i <= 10; i++) {  
        cout << i << " ";  
    }  
  
    cout << endl;  
    return 0;  
}
```

53.4.2 Ejercicio 2: tabla de multiplicar

Construya un programa que solicite un número y muestre su tabla de multiplicar del 1 al 10.

```
#include <iostream>
using namespace std;

int main() {
    int numero;

    cout << "Ingrese un numero: ";
    cin >> numero;

    for (int i = 1; i <= 10; i++) {
        cout << numero << " x " << i
            << " = " << numero * i << endl;
    }

    return 0;
}
```

💡 Pregunta para el grupo

¿Qué parte del ciclo habría que modificar para mostrar la tabla únicamente del 1 al 5?

53.5 Uso de enum

Un enum permite asignar nombres claros a un conjunto de valores relacionados.

```
enum Dia {
    LUNES = 1,
    MARTES,
    MIERCOLES,
    JUEVES,
    VIERNES
};
```

Internamente, los valores continúan siendo números, pero el código resulta más fácil de leer.

Analogía: en lugar de identificar las luces de un semáforo como 0, 1 y 2, podemos llamarlas ROJO, AMARILLO y VERDE.

```
#include <iostream>
using namespace std;

enum EstadoSemaforo {
    ROJO = 1,
```

```
    AMARILLO,  
    VERDE  
};  
  
int main() {  
    int opcion;  
  
    cout << "1. Rojo" << endl;  
    cout << "2. Amarillo" << endl;  
    cout << "3. Verde" << endl;  
    cout << "Seleccione el estado: ";  
    cin >> opcion;  
  
    switch (opcion) {  
        case ROJO:  
            cout << "Detenerse." << endl;  
            break;  
  
        case AMARILLO:  
            cout << "Prepararse para detenerse." << endl;  
            break;  
  
        case VERDE:  
            cout << "Puede avanzar." << endl;  
            break;  
  
        default:  
            cout << "Estado no valido." << endl;  
    }  
  
    return 0;  
}
```

53.5.1 Ventaja principal

Compare estas dos expresiones:

```
if (opcion == 3)
```

```
if (opcion == VERDE)
```

La segunda comunica mejor la intención del programa.

53.6 Proyecto del laboratorio: cajero automático

53.6.1 Situación

Un cajero automático presenta un menú y espera que el usuario seleccione una operación. Después de consultar, depositar o retirar, el menú vuelve a mostrarse. El programa termina únicamente cuando se selecciona **Salir**.

53.6.2 Operaciones requeridas

1. Consultar saldo.
2. Depositar dinero.
3. Retirar dinero.
4. Salir.

El saldo inicial será de **Q1,000.00**.

53.6.3 Reglas

- Los depósitos deben ser mayores que cero.
- Los retiros deben ser mayores que cero.
- No se puede retirar una cantidad superior al saldo disponible.
- El menú debe repetirse hasta seleccionar la opción de salida.
- Las opciones se representarán mediante `enum`.
- El menú se controlará con `switch`.
- La repetición se realizará con `do...while`.

53.7 4.1 ¿Por qué utilizar `do...while`?

El cajero debe mostrar el menú por lo menos una vez. Por ello, `do...while` es apropiado: primero ejecuta las instrucciones y luego evalúa si debe repetirlas.

```
do {  
    // instrucciones  
} while (condicion);
```

Analogía: al llegar a un cajero, primero se observa el menú. Después de realizar una operación se decide si se hará otra o si se terminará.

53.8 Diseño previo

Antes de programar, identifique las partes principales:

```
Inicio
  Definir saldo inicial
  Definir las opciones del menu

  Hacer
    Mostrar menu
    Leer opcion

    Segun la opcion
      Consultar saldo
      Depositar
      Retirar
      Salir
      Informar opcion invalida
  Mientras la opcion no sea Salir
Fin
```

53.9 Diagrama de flujo

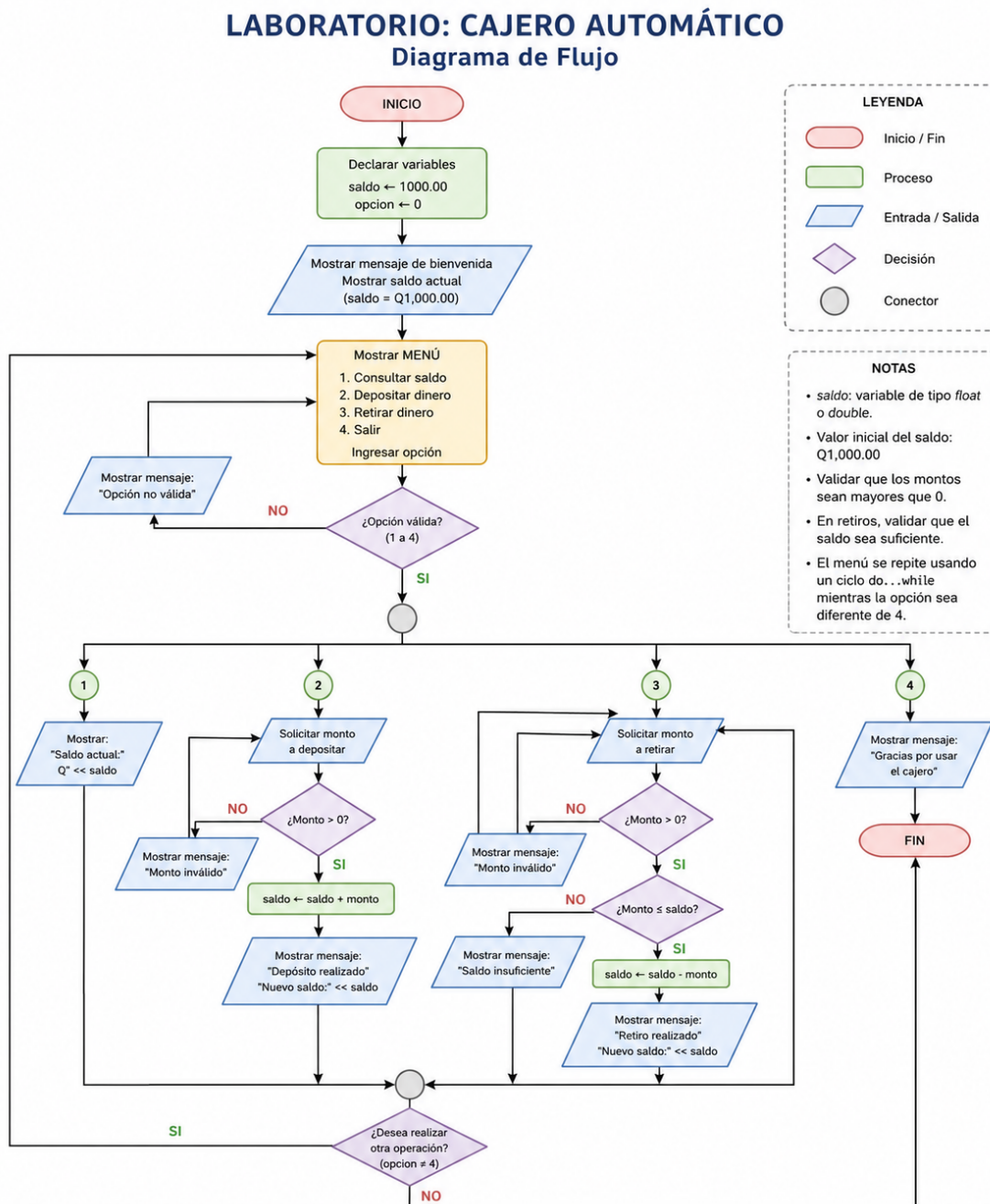


Figura 53.1: Diagrama de flujo del cajero automático

53.10 Definición de las opciones con enum

```
enum OpcionCajero {  
    CONSULTAR_SALDO = 1,  
    DEPOSITAR,  
    RETIRAR,  
    SALIR  
};
```

Como el primer valor es 1, los siguientes toman automáticamente los valores 2, 3 y 4.

53.11 Construcción paso a paso

53.11.1 Paso 1: variables principales

```
double saldo = 1000.00;  
double monto;  
int opcion;
```

53.11.2 Paso 2: mostrar el menú

```
cout << "\n==== CAJERO AUTOMATICO =====" << endl;  
cout << "1. Consultar saldo" << endl;  
cout << "2. Depositar dinero" << endl;  
cout << "3. Retirar dinero" << endl;  
cout << "4. Salir" << endl;  
cout << "Seleccione una opcion: ";  
cin >> opcion;
```

53.11.3 Paso 3: procesar la opción

```
switch (opcion) {  
    case CONSULTAR_SALDO:  
        cout << "Saldo disponible: Q" << saldo << endl;  
        break;  
  
    case DEPOSITAR:  
        // solicitar y validar el deposito
```

```
        break;

    case RETIRAR:
        // solicitar y validar el retiro
        break;

    case SALIR:
        cout << "Gracias por utilizar el cajero." << endl;
        break;

    default:
        cout << "Opcion no valida." << endl;
}

```

53.11.4 Paso 4: repetir el menú

```
do {
    // mostrar menu y procesar opcion
} while (opcion != SALIR);

```

53.12 5. Programa completo

```
#include <iostream>
#include <iomanip>
using namespace std;

enum OpcionCajero {
    CONSULTAR_SALDO = 1,
    DEPOSITAR,
    RETIRAR,
    SALIR
};

int main() {
    double saldo = 1000.00;
    double monto;
    int opcion;

    cout << fixed << setprecision(2);

```

```
do {
    cout << "\n==== CAJERO AUTOMATICO =====" << endl;
    cout << "1. Consultar saldo" << endl;
    cout << "2. Depositar dinero" << endl;
    cout << "3. Retirar dinero" << endl;
    cout << "4. Salir" << endl;
    cout << "Seleccione una opcion: ";
    cin >> opcion;

    switch (opcion) {
        case CONSULTAR_SALDO:
            cout << "Saldo disponible: Q" << saldo << endl;
            break;

        case DEPOSITAR:
            cout << "Ingrese el monto a depositar: Q";
            cin >> monto;

            if (monto > 0) {
                saldo = saldo + monto;
                cout << "Deposito realizado correctamente." << endl;
                cout << "Nuevo saldo: Q" << saldo << endl;
            } else {
                cout << "El monto debe ser mayor que cero." << endl;
            }
            break;

        case RETIRAR:
            cout << "Ingrese el monto a retirar: Q";
            cin >> monto;

            if (monto <= 0) {
                cout << "El monto debe ser mayor que cero." << endl;
            } else if (monto > saldo) {
                cout << "Fondos insuficientes." << endl;
            } else {
                saldo = saldo - monto;
                cout << "Retiro realizado correctamente." << endl;
                cout << "Nuevo saldo: Q" << saldo << endl;
            }
            break;

        case SALIR:
            cout << "Gracias por utilizar el cajero automatico." << endl;
            break;

        default:
```

```

        cout << "Opcion no valida. Intente nuevamente." <<
    }

    } while (opcion != SALIR);

    return 0;
}

```

53.13 Pruebas del programa

Antes de considerar terminado el ejercicio, pruebe por lo menos los siguientes casos:

Prueba	Acción	Resultado esperado
1	Consultar al iniciar	Mostrar Q1,000.00
2	Depositar Q500.00	Nuevo saldo Q1,500.00
3	Retirar Q300.00	Nuevo saldo Q1,200.00
4	Retirar más que el saldo	Mostrar fondos insuficientes
5	Depositar un número negativo	Rechazar el monto
6	Seleccionar una opción inexistente	Mostrar opción no válida
7	Seleccionar Salir	Finalizar el ciclo

! Idea clave

Un programa no está terminado solamente porque compila. También debe probarse con datos correctos, incorrectos y casos límite.

53.14 Reto opcional

Cuando el programa básico funcione, agregue **una** de las siguientes mejoras:

- solicitar un PIN antes de mostrar el menú;
- contar cuántas operaciones se realizaron;
- establecer un retiro máximo por operación;
- mostrar un mensaje cuando el saldo sea menor que Q100.00.

53.15 Cierre

En el cajero automático se combinaron varias estructuras:

- `enum` dio nombres a las opciones;
- `switch` seleccionó la operación;
- `if` validó depósitos y retiros;

- `do...while` mantuvo activo el menú;
- las variables almacenaron el saldo y los montos.

El valor del ejercicio no está únicamente en hacer funcionar un cajero, sino en comprender cómo varias estructuras pequeñas pueden integrarse para resolver un problema completo.

54 Capítulo 5. Integración de conceptos: construyendo un programa completo

Caso de estudio: Sistema de Control de Parqueos

55 Introducción

Este capítulo integra los conceptos estudiados hasta el momento para resolver un problema completo en C++.

56 Objetivos

- Integrar variables, operadores y estructuras de control.
- Analizar un problema antes de programar.
- Diseñar una solución mediante pseudocódigo y Mermaid.

57 Caso de estudio

57.1 Sistema de Control de Parqueos

58 Análisis del problema

Entradas	Procesos	Salidas
Nombre	Determinar tarifa	Total
Placa	Calcular	Resumen
Tipo	Validar	Mensajes
Horas	Operaciones	Información

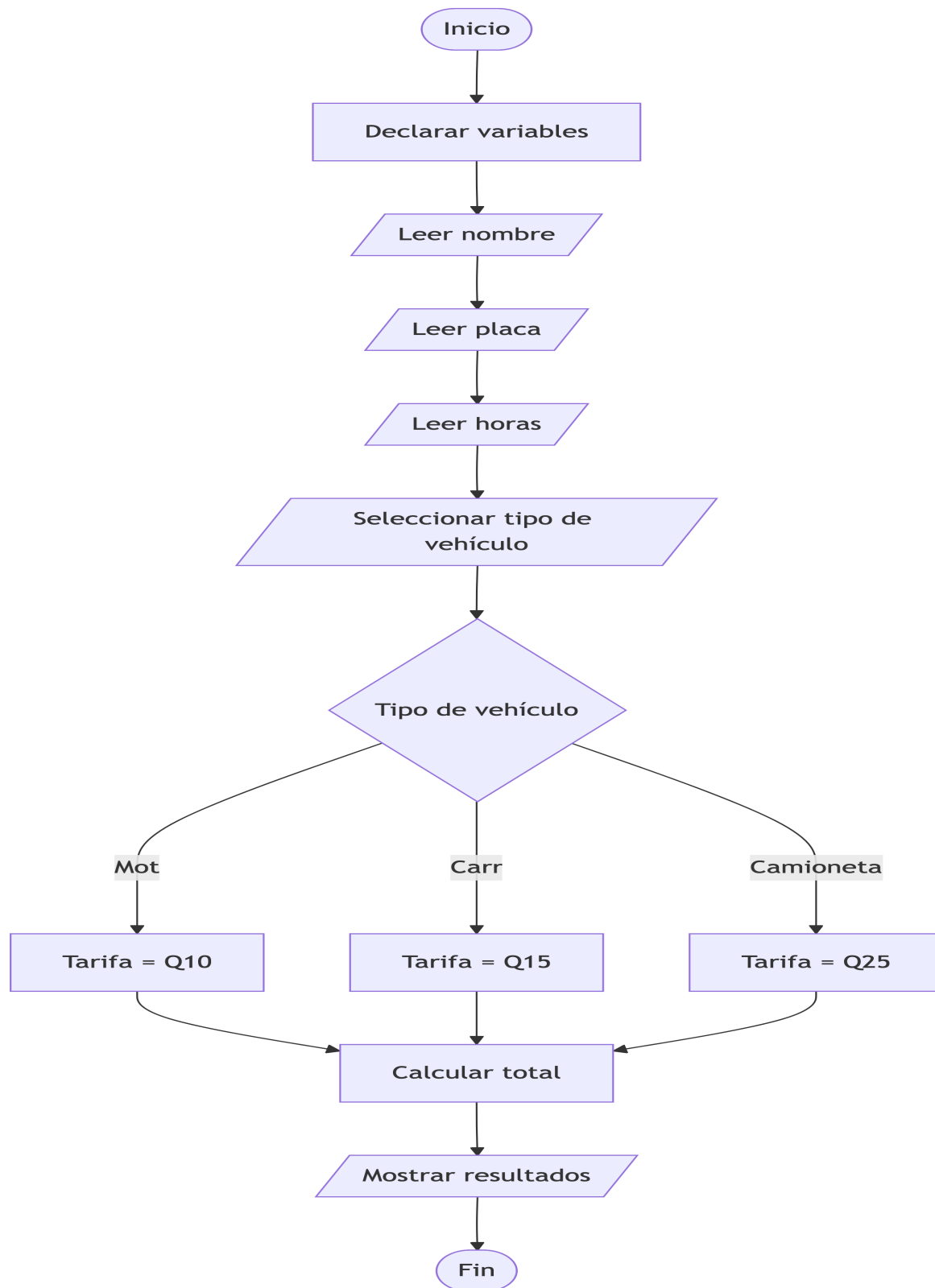
59 Variables

Variable	Tipo
nombreConductor	string
placa	string
tipoVehiculo	int
horas	int
tarifa	double
totalPagar	double

60 Pseudocódigo

```
Inicio  
  Declarar variables  
  Leer datos  
  Determinar tarifa  
  Calcular total  
  Mostrar resultados  
Fin
```


61 Diagrama: Sistema de control de parqueos



62 Paso 1

```
#include <iostream>
#include <string>
using namespace std;
```


63 Paso 2

```
//-----  
// Enumeración de los tipos de vehículos  
//-----  
  
enum TipoVehiculo  
{  
    MOTO = 1,  
    CARRO,  
    CAMIONETA  
};  
  
//-----  
// Declaración de variables  
//-----  
  
// En esta sección declaramos todas las variables que  
// utilizaremos durante el programa, aunque algunas  
// todavía no se empleen en los pasos siguientes.  
// Esto mejora la organización y facilita la lectura  
// del código.  
  
string nombreConductor;  
string placa;  
  
int tipoVehiculo;  
int horas;  
  
double tarifa = 0.0;  
double totalPagar = 0.0;
```


64 Paso 3. Entrada

```
cout<<"Nombre: ";
getline(cin,nombreConductor);
cout<<"Placa: ";
getline(cin,placa);

cout<<"Horas: ";
cin>>horas;

cout << "Tipo de vehículo" << endl;
cout << "1. Moto" << endl;
cout << "2. Carro" << endl;
cout << "3. Camioneta" << endl;
cout << "\nSeleccione una opción: ";
cin >> tipoVehiculo;
```


65 Paso 4. Tarifa

```
switch(tipoVehiculo){  
case MOTO: tarifa=10; break;  
case CARRO: tarifa=15; break;  
case CAMIONETA: tarifa=25; break;  
default: tarifa=0;  
}
```


66 Paso 5

```
totalPagar=tarifa*horas;
```


67 Paso 6

```
cout<<"Total: Q"<<totalPagar<<endl;
```


68 Reflexión

- ¿Por qué tarifa es double?
- ¿Qué hace switch?

69 Resumen

Tema	Aplicado
Variables	☐
Operadores	☐
switch	☐
if	☐
while	☐
do...while	☐
for	☐

70 Paso 7. Mostrando los resultados

Una vez calculado el total a pagar, es momento de presentar la información al usuario de forma clara y ordenada.

```
//-----  
//  Mostrar resultados  
//-----  
  
cout << "\n===== \n";  
cout << "          SISTEMA DE CONTROL DE PARQUEOS\n";  
cout << "===== \n\n";  
  
cout << "Conductor : " << nombreConductor << endl;  
cout << "Placa      : " << placa << endl;  
  
cout << "Tipo de vehículo" << endl;  
cout << "1. Moto" << endl;  
cout << "2. Carro" << endl;  
cout << "3. Camioneta" << endl;  
cout << "\nSeleccione una opción: ";  
cin >> tipoVehiculo;  
  
switch(tipoVehiculo)  
{  
    case MOTO: cout << "Moto"; break;  
    case CARRO: cout << "Carro"; break;  
    case CAMIONETA: cout << "Camioneta"; break;  
}  
  
cout << endl;  
cout << "Horas      : " << horas << endl;  
cout << "Tarifa     : Q" << tarifa << endl;  
cout << "-----" << endl;  
cout << "TOTAL A PAGAR : Q" << totalPagar << endl;
```

70.1 Ejemplo de ejecución

Entrada

```
Nombre: Jorge García  
Placa: P123ABC  
Tipo de vehículo: 2  
Horas: 3
```

Salida

```
=====
          SISTEMA DE CONTROL DE PARQUEOS
=====

Conductor  : Jorge García
Placa      : P123ABC
Vehículo   : Carro
Horas      : 3
Tarifa     : Q15

-----
TOTAL A PAGAR : Q45
```

70.2 Reflexionemos

- ¿Por qué es importante presentar la información de forma ordenada?
- ¿Qué ventajas tiene mostrar el nombre del vehículo en lugar del número seleccionado?
- ¿Qué parte del programa realiza el cálculo y cuál solamente muestra información?

71 Paso 8. Analizando el programa

Concepto	¿Dónde aparece?
Variables	Declaración de variables
Operadores	<code>tarifa * horas</code>
<code>switch</code>	Selección de la tarifa
<code>cout</code>	Presentación de resultados

Hasta este punto ya tenemos un programa completamente funcional.

72 Paso 8. ¿Cómo podemos mejorarlo?

Nuestro programa funciona correctamente, pero como ocurre en el desarrollo de software real, siempre podemos incorporar nuevas características.

Cada mejora será la oportunidad perfecta para recordar una estructura de control ya estudiada.

Necesidad del programa	Concepto que utilizaremos
Validar las horas ingresadas	<code>if</code>
Clasificar el tiempo de permanencia	<code>if...else</code>
Aplicar descuentos	<code>if...else if</code>
Registrar varios vehículos	<code>while</code>
Mostrar un menú principal	<code>do...while</code>
Mostrar los espacios disponibles	<code>for</code>

72.1 Lo que construiremos a continuación

En la siguiente versión del programa incorporaremos:

1. Validación de datos.
2. Descuentos según el tiempo de permanencia.
3. Registro de varios vehículos.
4. Menú principal.
5. Visualización de espacios del parqueo.

Recuerda

Un buen programador no memoriza estructuras de control.
Las utiliza para resolver problemas reales.

72.2 Preparándonos para la siguiente clase

En la próxima sesión reorganizaremos este mismo programa utilizando:

- Funciones
- Procedimientos
- Parámetros

El objetivo será comprobar cómo un programa que ya funciona puede organizarse de una manera más profesional.

73 Capítulo 6. Organizando nuestros programas mediante funciones y procedimientos

73.1 Caso de estudio

73.1.1 Evolucionando el Sistema de Control de Parqueos

74 Introducción

En el capítulo anterior desarrollamos un programa completo para calcular el cobro de un parqueo. El programa resolvía correctamente el problema planteado y nos permitió integrar variables, operadores y estructuras de control.

Ahora el programa funciona, pero ha crecido. Cuando un programa aumenta de tamaño también aumenta la dificultad para leerlo, comprenderlo y modificarlo.

En el desarrollo profesional del software es poco común escribir todo dentro de `main()`. En su lugar, el programa se divide en pequeños módulos donde cada uno realiza una única responsabilidad.

75 Objetivos

- Comprender por qué organizamos un programa.
- Diferenciar funciones y procedimientos.
- Comprender el uso de parámetros.
- Reorganizar el Sistema de Control de Parqueos.

76 ¿Por qué necesitamos organizar nuestros programas?

Antes de escribir una sola línea de código, observa nuevamente el programa de la clase anterior.

 Pregunta para discutir

¿Les parece fácil leer un programa donde todo está escrito dentro del `main()` ?

Permite que los estudiantes expresen sus ideas y luego introduce el concepto de modularidad.

77 La analogía de la construcción de una casa

Una casa no es construida por una sola persona.

Profesional	Responsabilidad
Arquitecto	Diseñar la casa
Albañil	Construir la estructura
Electricista	Instalar el sistema eléctrico
Plomero	Instalar las tuberías
Pintor	Realizar el acabado final

Cada uno tiene una única responsabilidad.

Un programa bien diseñado funciona exactamente igual: cada módulo realiza una tarea específica.

78 Procedimientos y funciones

79 ¿Qué es un procedimiento?

Un procedimiento es un bloque de código que realiza una tarea específica y **no devuelve un valor**.

Las siguientes tareas de nuestro sistema pueden convertirse en procedimientos:

- Mostrar el título.
- Leer información.
- Mostrar resultados.

Recuerda

Un procedimiento ejecuta acciones.

80 ¿Qué es una función?

Algunas tareas necesitan calcular un resultado.

Por ejemplo:

- Determinar la tarifa.
- Calcular el total a pagar.

En estos casos utilizaremos una función.

Una función realiza una tarea específica y devuelve un valor al programa que la llamó.

81 Procedimiento o función

Característica	Procedimiento	Función
Realiza una tarea	☐	☐
Devuelve un valor	☐	☐
Utiliza <code>return</code>	No	Sí

Reflexiona

¿Cuáles tareas del Sistema de Control de Parqueos implementarías mediante procedimientos y cuáles mediante funciones?

82 ¿Qué son los parámetros?

Una función sabe **qué debe hacer**, pero necesita información para poder trabajar.
A esa información la llamamos **parámetros**.

83 La analogía de la casa

Cada trabajador necesita materiales para realizar su trabajo.

Profesional	¿Qué necesita?
Electricista	Cables e interruptores
Albañil	Ladrillos y cemento
Pintor	Pintura

De la misma manera, una función necesita recibir datos para realizar su tarea.

Un parámetro es un dato que se envía a una función o procedimiento para que pueda realizar su trabajo.

i Idea importante

Las funciones siempre realizan la misma tarea.
Lo único que cambia son los datos que reciben.

La siguiente sección transformará paso a paso nuestro Sistema de Control de Parques utilizando procedimientos, funciones y parámetros.

84 Transformando nuestro programa

Hasta este momento ya conocemos los conceptos de **procedimientos, funciones y parámetros**.

Ahora llegó el momento de aplicarlos sobre el programa desarrollado en la clase anterior.

Nuestro objetivo **no es cambiar el funcionamiento del Sistema de Control de Parqueos**.

El objetivo es **organizar mejor el código**.

Al finalizar esta sección, el programa realizará exactamente las mismas tareas, pero será mucho más fácil de leer, comprender y mantener.

85 Primera mejora: mover el título del programa

85.1 Objetivo

Crear un procedimiento que sea el único responsable de mostrar el encabezado del sistema.

Piensa

Si mañana deseamos cambiar el nombre del sistema o mejorar el diseño del encabezado, ¿en cuántos lugares tendríamos que modificar el programa?

Después de crear el procedimiento, elimina ese código del `main()` y reemplázalo por una única llamada al procedimiento.

Analiza el resultado.

El programa continúa funcionando exactamente igual.

Sin embargo, el `main()` ya es más corto y fácil de leer.

86 Segunda mejora: separar la lectura de datos

86.1 Objetivo

Crear un procedimiento encargado de solicitar toda la información al usuario.

Este procedimiento será responsable de:

- Leer el nombre del cliente.
- Leer el tipo de vehículo.
- Leer la cantidad de horas.

Una vez terminado, el `main()` únicamente deberá llamar al procedimiento.

Observa cómo comienza a desaparecer el código repetitivo del programa principal.

87 Tercera mejora: crear una función para determinar la tarifa

Hasta este momento el programa determina la tarifa utilizando una estructura de decisión. Ahora esa responsabilidad será trasladada a una función.

87.1 Objetivo

Crear una función que reciba el tipo de vehículo y devuelva la tarifa correspondiente.

El programa principal ya no necesitará conocer cómo se calcula la tarifa.

Únicamente solicitará el resultado.

i Importante

Una buena función oculta los detalles del cálculo.
El programa principal solamente utiliza el resultado obtenido.

88 Cuarta mejora: crear una función para calcular el total

Una vez conocida la tarifa, podemos crear otra función cuya única responsabilidad sea calcular el total a pagar.

Esta función recibirá:

- La tarifa.
- La cantidad de horas.

Y devolverá el monto total.

Observa que ahora cada función tiene una única responsabilidad.

89 Quinta mejora: crear un procedimiento para mostrar el reporte

Finalmente trasladaremos toda la salida del programa a un procedimiento.

Este procedimiento mostrará:

- Nombre del cliente.
- Tipo de vehículo.
- Horas.
- Tarifa.
- Total a pagar.

El programa principal dejará de preocuparse por la presentación de la información.

Su única responsabilidad será coordinar la ejecución del programa.

90 Nuestro nuevo programa

Al finalizar las mejoras, el `main()` será mucho más pequeño y mucho más fácil de comprender. Su lectura se parece más a una lista de tareas que a un bloque de instrucciones.



💡 Reflexiona

Observa el diagrama anterior.
Cada bloque realiza una única responsabilidad.

Esta es una de las principales características de un programa bien organizado.

91 ¿Qué ganamos con esta organización?

Al reorganizar nuestro programa obtenemos varias ventajas:

- El código es más fácil de leer.
- Cada módulo tiene una responsabilidad específica.
- Las modificaciones futuras serán más sencillas.
- Es posible reutilizar funciones en otros programas.
- El trabajo en equipo resulta más simple, ya que diferentes programadores pueden trabajar sobre módulos distintos.

En el siguiente apartado compararemos la primera versión del programa con esta nueva versión organizada para identificar las mejoras obtenidas.

92 Primera modificación: crear el procedimiento `mostrarTitulo()`

Hasta este momento, el encabezado del sistema se encuentra escrito directamente dentro de `main()`.

El primer cambio será mover únicamente esas instrucciones a un procedimiento independiente.

93 1. Identificamos el código que vamos a mover

Localiza dentro de `main()` las siguientes instrucciones:

```
cout << "\n===== \n";  
cout << "    SISTEMA DE CONTROL DE PARQUEOS \n";  
cout << "===== \n \n";
```

Estas líneas cumplen una sola responsabilidad: mostrar el título del sistema.

94 2. Creamos el procedimiento

Antes de `main()`, escribe:

```
//-----  
// Muestra el encabezado del sistema  
//-----  
void mostrarTitulo()  
{  
    cout << "\n=====\\n";  
    cout << "    SISTEMA DE CONTROL DE PARQUEOS\\n";  
    cout << "=====\\n\\n";  
}
```

La palabra `void` indica que el procedimiento realiza una acción, pero no devuelve ningún valor.

95 3. Modificamos `main()`

Elimina del `main()` las tres instrucciones originales y reemplázalas por:

```
mostrarTitulo();
```


96 4. Código después de la primera modificación

```
#include <iostream>
#include <string>

using namespace std;

//-----
// Enumeración de los tipos de vehículos
//-----
enum TipoVehiculo
{
    MOTO = 1,
    CARRO,
    CAMIONETA
};

//-----
// Declaración de variables
//-----
string nombreConductor;
string placa;

int tipoVehiculo;
int horas;

double tarifa = 0.0;
double totalPagar = 0.0;

//-----
// Muestra el encabezado del sistema
//-----
void mostrarTitulo()
{
    cout << "\n===== \n";
    cout << "    SISTEMA DE CONTROL DE PARQUEOS \n";
}
```

```
        cout << "=====\n\n";
    }

    int main()
    {
        cout << "Nombre: ";
        getline(cin, nombreConductor);

        cout << "Placa: ";
        getline(cin, placa);

        cout << "Horas: ";
        cin >> horas;

        cout << "\nTipo de vehículo" << endl;
        cout << "1. Moto" << endl;
        cout << "2. Carro" << endl;
        cout << "3. Camioneta" << endl;
        cout << "\nSeleccione una opción: ";
        cin >> tipoVehiculo;

        switch (tipoVehiculo)
        {
            case MOTO:
                tarifa = 10;
                break;

            case CARRO:
                tarifa = 15;
                break;

            case CAMIONETA:
                tarifa = 25;
                break;

            default:
                tarifa = 0;
        }

        totalPagar = tarifa * horas;

        mostrarTitulo();

        cout << "Conductor : " << nombreConductor << endl;
        cout << "Placa      : " << placa << endl;

        cout << "Vehículo   : ";
```

```
switch (tipoVehiculo)
{
    case MOTO:
        cout << "Moto";
        break;

    case CARRO:
        cout << "Carro";
        break;

    case CAMIONETA:
        cout << "Camioneta";
        break;

    default:
        cout << "No válido";
}

cout << endl;
cout << "Horas      : " << horas << endl;
cout << "Tarifa      : Q" << tarifa << endl;
cout << "-----" << endl;
cout << "TOTAL A PAGAR: Q" << totalPagar << endl;

return 0;
}
```

Comprobación

Compila y ejecuta el programa.
La salida debe ser exactamente la misma que antes.

¿Qué cambió?

El encabezado ya no está escrito directamente dentro de `main()`.
Ahora esa tarea pertenece al procedimiento `mostrarTitulo()`.

97 Segunda modificación: crear el procedimiento `LeerDatos()`

En la modificación anterior organizamos el encabezado del sistema.

Ahora haremos lo mismo con la entrada de datos.

Nuestro objetivo será que **todo el proceso de captura de información** quede dentro de un solo procedimiento.

98 1. Identificamos el código que vamos a mover

Dentro de `main()` localiza todas las instrucciones utilizadas para leer:

- Nombre del conductor.
- Placa.
- Horas.
- Tipo de vehículo.

Todo ese bloque pertenece a una misma responsabilidad: **leer los datos del usuario.**

99 2. Creamos el procedimiento

Antes de `main()` agrega el siguiente procedimiento:

```
//-----  
// Solicita la información del conductor  
//-----  
void leerDatos()  
{  
    cout << "Nombre: ";  
    getline(cin, nombreConductor);  
  
    cout << "Placa: ";  
    getline(cin, placa);  
  
    cout << "Horas: ";  
    cin >> horas;  
  
    cout << "\nTipo de vehículo" << endl;  
    cout << "1. Moto" << endl;  
    cout << "2. Carro" << endl;  
    cout << "3. Camioneta" << endl;  
    cout << "\nSeleccione una opción: ";  
    cin >> tipoVehiculo;  
}
```

Observa que el procedimiento únicamente realiza una tarea: solicitar información al usuario.

100 3. Modificamos `main()`

Ahora elimina todas las instrucciones de entrada de datos y reemplázalas por una sola línea:

```
leerDatos();
```

El inicio del programa principal ahora queda mucho más limpio.

```
int main()
{
    leerDatos();

    // El resto del programa continúa aquí...
}
```


101 4. ¿Qué ganamos?

Comparemos ambas versiones.

101.1 Antes

```
main()  
├ Leer nombre  
├ Leer placa  
├ Leer horas  
├ Mostrar menú  
├ Leer tipo de vehículo  
...
```

101.2 Después

```
main()  
├ leerDatos()  
...
```

El programa sigue haciendo exactamente lo mismo, pero ahora el nombre del procedimiento describe claramente la tarea que realiza.

Comprobación

Compila nuevamente el programa.
Si todo está correcto, el comportamiento será exactamente el mismo que antes.

¿Qué aprendimos?

Un procedimiento puede agrupar varias instrucciones relacionadas en una sola unidad lógica. Esto hace que el programa sea más fácil de leer y mantener.

101.3 Preparándonos para la siguiente modificación

En la siguiente sección moveremos el `switch` encargado de determinar la tarifa hacia una **función**.

Será la primera vez que nuestro programa devolverá un valor utilizando `return`.

102 Tercera modificación: crear la función `determinarTarifa()`

Hasta este momento nuestro programa utiliza un bloque `switch` dentro de `main()` para determinar la tarifa según el tipo de vehículo.

En esta modificación moveremos esa responsabilidad hacia una función.

103 1. Identificamos el código que vamos a mover

Localiza el siguiente bloque dentro de `main()`:

```
switch (tipoVehiculo)
{
    case MOTO:
        tarifa = 10;
        break;

    case CARRO:
        tarifa = 15;
        break;

    case CAMIONETA:
        tarifa = 25;
        break;

    default:
        tarifa = 0;
}
```

Observa que este bloque realiza una única tarea: **determinar la tarifa**.

104 2. Creamos la función

Antes de `main()` agrega la siguiente función:

```
//-----  
// Determina la tarifa según el tipo de vehículo  
//-----  
double determinarTarifa(int tipo)  
{  
    switch (tipo)  
    {  
        case MOTO:  
            return 10;  
  
        case CARRO:  
            return 15;  
  
        case CAMIONETA:  
            return 25;  
  
        default:  
            return 0;  
    }  
}
```

104.1 Analicemos el encabezado

```
double determinarTarifa(int tipo)
```

Elemento	Significado
<code>double</code>	La función devolverá un valor decimal.
<code>determinarTarifa</code>	Nombre de la función.
<code>int tipo</code>	Parámetro que recibe el tipo de vehículo.

A diferencia de un procedimiento, esta función **devuelve un resultado** mediante la instrucción `return`.

105 3. Modificamos `main()`

Elimina el bloque `switch` y reemplázalo por una única línea:

```
tarifa = determinarTarifa(tipoVehiculo);
```

Lee esa instrucción en voz alta:

“La variable **tarifa** recibirá el valor que devuelva la función **determinarTarifa()**.”

106 4. ¿Qué sucede internamente?

```
main()
  ↓ envía tipoVehiculo
determinarTarifa()
  ↓ calcula la tarifa
return tarifa
  ↓
main() guarda el resultado en la variable tarifa
```



107 5. ¿Qué ganamos?

107.1 Antes

```
main()  
├ switch(...)  
├ case MOTO  
├ case CARRO  
└ case CAMIONETA
```

107.2 Después

```
main()  
└ tarifa = determinarTarifa(tipoVehiculo);
```

Ahora el programa principal ya no necesita conocer cómo se calcula la tarifa.

Esa responsabilidad quedó encapsulada dentro de la función.

Comprobación

Compila y ejecuta nuevamente el programa.
El resultado debe ser exactamente el mismo que en las modificaciones anteriores.

¿Qué aprendimos?

Una función permite ocultar los detalles de un cálculo.
El programa principal únicamente solicita el resultado y continúa con la siguiente tarea.

107.3 Preparándonos para la siguiente modificación

En la siguiente modificación construiremos la función `calcularTotal()`.

Verás que el `main()` comienza a parecer una lista de tareas de alto nivel.

108 Cuarta modificación: crear la función `calcularTotal()`

En la modificación anterior logramos que el programa determinara la tarifa utilizando una función. Ahora haremos exactamente lo mismo con el cálculo del total a pagar.

109 1. Identificamos el código que vamos a mover

Dentro de `main()` localiza la siguiente instrucción:

```
totalPagar = tarifa * horas;
```

Aunque únicamente es una línea de código, representa una responsabilidad muy importante del programa: **calcular el monto que deberá pagar el cliente.**

110 2. Creamos la función

Antes de `main()` agrega la siguiente función:

```
//-----  
// Calcula el total a pagar  
//-----  
double calcularTotal(double tarifaHora, int cantidadHoras)  
{  
    return tarifaHora * cantidadHoras;  
}
```

Observa que esta función recibe dos parámetros:

Parámetro	Descripción
<code>tarifaHora</code>	Tarifa correspondiente al vehículo
<code>cantidadHoras</code>	Horas que permaneció estacionado

Después realiza el cálculo y devuelve el resultado mediante `return`.

111 3. Modificamos main ()

Elimina la instrucción original y reemplázala por:

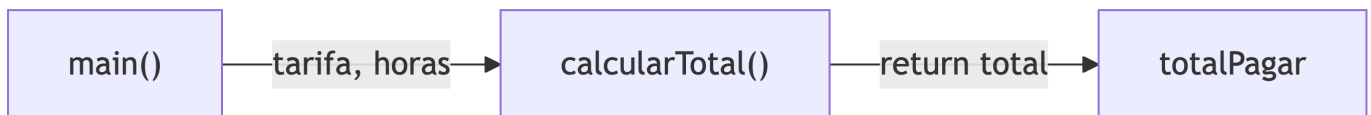
```
totalPagar = calcularTotal(tarifa, horas);
```

Lee la instrucción como si fuera una frase:

“TotalPagar recibirá el valor que devuelva la función calcularTotal () utilizando la tarifa y las horas.”

112 4. ¿Qué sucede internamente?

```
main()
  ↓ envía tarifa y horas
calcularTotal()
  ↓ realiza el cálculo
return total
  ↓
main() guarda el resultado
```



113 5. ¿Qué ganamos?

113.1 Antes

```
main()
├─ tarifa = determinarTarifa(...)
├─ totalPagar = tarifa * horas
└─ mostrar resultados
```

113.2 Después

```
main()
├─ tarifa = determinarTarifa(...)
├─ totalPagar = calcularTotal(...)
└─ mostrar resultados
```

Ahora el programa principal ya no realiza cálculos directamente.

Únicamente coordina la ejecución de las funciones.

Comprobación

Compila nuevamente el programa.
La salida debe ser exactamente igual que en las versiones anteriores.

¿Qué aprendimos?

Cada función debe tener una única responsabilidad.
`determinarTarifa()` determina la tarifa.
`calcularTotal()` calcula el total.
Separar responsabilidades hace que el programa sea más fácil de comprender y mantener.

113.3 Preparándonos para la siguiente modificación

Solo nos queda un paso.

Moveremos toda la presentación de resultados al procedimiento `mostrarResultados()`.

Cuando terminemos, el `main()` quedará reducido a unas pocas líneas y describirá claramente el flujo del programa.

114 Quinta modificación: crear el procedimiento `mostrarResultados()`

Ya organizamos la entrada de datos y los cálculos del programa.

Solo nos queda una última responsabilidad: presentar la información al usuario.

En esta modificación moveremos todo el reporte final hacia un procedimiento independiente.

115 1. Identificamos el código que vamos a mover

Localiza dentro de `main()` todas las instrucciones utilizadas para mostrar:

- El encabezado.
- El nombre del conductor.
- La placa.
- El tipo de vehículo.
- Las horas.
- La tarifa.
- El total a pagar.

Todo ese bloque tiene una única responsabilidad: **mostrar el resultado del proceso.**

116 2. Creamos el procedimiento

Antes de `main()` agrega el siguiente procedimiento.

```
//-----  
// Muestra el reporte final  
//-----  
void mostrarResultados()  
{  
    mostrarTitulo();  
  
    cout << "Conductor : " << nombreConductor << endl;  
    cout << "Placa      : " << placa << endl;  
  
    cout << "Vehículo  : ";  
  
    switch (tipoVehiculo)  
    {  
        case MOTO:  
            cout << "Moto";  
            break;  
  
        case CARRO:  
            cout << "Carro";  
            break;  
  
        case CAMIONETA:  
            cout << "Camioneta";  
            break;  
  
        default:  
            cout << "No válido";  
    }  
  
    cout << endl;  
    cout << "Horas      : " << horas << endl;  
    cout << "Tarifa     : Q" << tarifa << endl;  
    cout << "-----" << endl;
```

```
    cout << "TOTAL A PAGAR: Q" << totalPagar << endl;  
}
```

Observa algo interesante.

Dentro de este procedimiento estamos llamando nuevamente a:

```
mostrarTitulo();
```

Esto demuestra que un procedimiento puede utilizar otro procedimiento.

117 3. Modificamos `main()`

Elimina todo el bloque de salida y reemplázalo por una sola instrucción.

```
mostrarResultados();
```


118 4. Nuestro nuevo main()

Después de todas las modificaciones, el programa principal queda reducido a unas pocas líneas.

```
int main()
{
    leerDatos();

    tarifa = determinarTarifa(tipoVehiculo);

    totalPagar = calcularTotal(tarifa, horas);

    mostrarResultados();

    return 0;
}
```

Ahora el `main()` puede leerse casi como un algoritmo escrito en lenguaje natural.

1. Leer datos.
2. Determinar la tarifa.
3. Calcular el total.
4. Mostrar resultados.

119 5. ¿Qué cambió?

Comparemos ambas versiones.

119.1 Antes

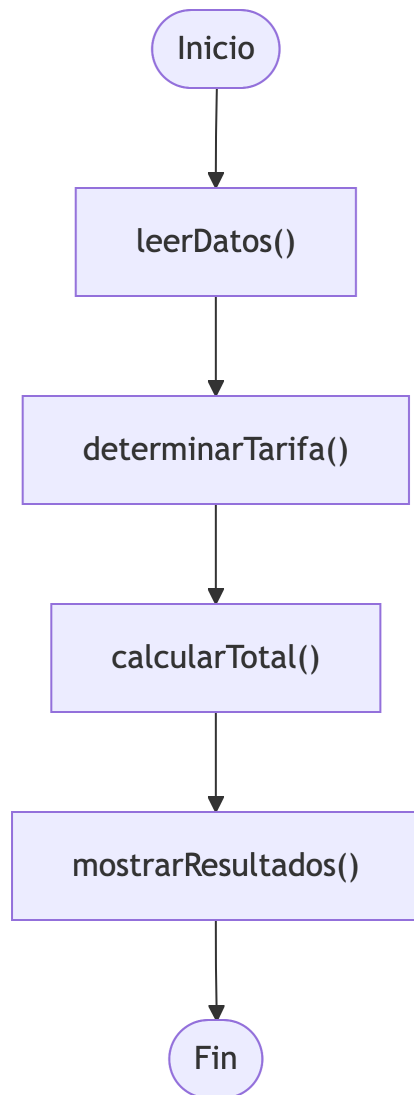
```
main()  
├ Leer datos  
├ Determinar tarifa  
├ Calcular total  
├ Mostrar encabezado  
├ Mostrar conductor  
├ Mostrar vehículo  
├ Mostrar horas  
├ Mostrar tarifa  
└ Mostrar total
```

119.2 Después

```
main()  
├ leerDatos()  
├ determinarTarifa()  
├ calcularTotal()  
└ mostrarResultados()
```

Observa que el programa realiza exactamente las mismas tareas.

La diferencia es que ahora cada responsabilidad fue trasladada a un módulo independiente.



120 Comprobación final

Compila nuevamente el programa.

Utiliza los mismos datos empleados en el capítulo anterior.

El resultado obtenido debe ser exactamente el mismo.

Si el comportamiento no cambió, significa que la reorganización fue realizada correctamente.

Felicitaciones

Has transformado un programa funcional en un programa mucho mejor organizado.
Eso es precisamente lo que ocurre durante el desarrollo profesional del software.

121 Lo que aprendimos

Durante este capítulo reorganizamos un programa completo utilizando procedimientos, funciones y parámetros.

Responsabilidad	Módulo
Mostrar el encabezado	<code>mostrarTitulo()</code>
Leer información	<code>leerDatos()</code>
Determinar la tarifa	<code>determinarTarifa()</code>
Calcular el total	<code>calcularTotal()</code>
Mostrar resultados	<code>mostrarResultados()</code>

Observa que cada módulo tiene una única responsabilidad.

Gracias a ello, el programa principal quedó reducido a unas pocas líneas fáciles de leer.

Idea clave

Programar mejor no significa escribir más código.
Programar mejor significa organizar el código de manera que cualquier persona pueda comprenderlo y mantenerlo.

122 Preparándonos para el siguiente capítulo

Nuestro Sistema de Control de Parques ya está organizado.

En el siguiente capítulo comenzaremos a trabajar con **arreglos**, lo que permitirá registrar y administrar la información de varios vehículos en lugar de procesar únicamente uno.

123 Cierre del capítulo

Consejo final del Ingeniero

Un buen programador no es quien escribe código más rápido, sino quien piensa mejor la solución antes de programarla.

123.1 Próximo paso

En el siguiente capítulo continuaremos fortaleciendo la lógica mediante pseudocódigo, diagramas y ejercicios guiados.

